

Neural Sentinel: Decentralized, Deep Learning based Intrusion Detection Solution

Submitted in partial fulfillment of the requirements
of the degree of

BACHELOR OF ENGINEERING

In

COMPUTER ENGINEERING

By

Yash Shelar

Gauresh Bagayatkar

Vedant Chaudhari

Ocean Chaudhary

Supervisor:

Mr Vaibhav Ambhire

(Professor, Department of Computer Engineering, TSEC)



Computer Engineering Department
Thadomal Shahani Engineering College
University of Mumbai
2025 – 2026

Neural Sentinel: Decentralized, Deep learning based Intrusion Detection Solution

Submitted in partial fulfillment of the requirements
of the degree of

BACHELOR OF ENGINEERING

In

COMPUTER ENGINEERING

By

Group No: 4

2203153	Yash Shelar
2313194	Gauresh Bagayatkar
2313211	Ocean Chaudhary
2313212	Vedant Chaudhari

Supervisor:

Mr Vaibhav Ambhire
(Professor, Department of Computer Engineering, TSEC)



Computer Engineering Department
Thadomal Shahani Engineering College
University of Mumbai
2025 - 2026

CERTIFICATE

This is to certify that the project entitled “Neural Sentinel: Decentralized, Deep learning-based intrusion detection solution” is a bonafide work of

2203153	Yash Shelar
2313194	Gauresh Bagayatkar
2313211	Ocean Chaudhary
2313212	Vedant Chaudhari

Submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of **“BACHELOR OF ENGINEERING”** in **“COMPUTER ENGINEERING”**.

Mr Vaibhav Ambhire
Supervisor/Guide

Dr Tanuja Sarode
Head of Department

Dr. G.T. Thampi
Principal

Project Report Approval for B.E

Project report entitled *Neural Sentinel: Decentralized, Deep learning based intrusion detection solution* by

2203153

Yash Shelar

2313194

Gauresh Bagayatkar

2313211

Ocean Chaudhary

2313212

Vedant Chaudhari

is approved for the degree of ***“BACHELOR OF ENGINEERING”*** in
“COMPUTER ENGINEERING”.

Examiners

1. _____

2. _____

Date:

Place:

Declaration

I declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

1) _____

Yash Shelar 2203153

2) _____

Gauresh Bagayatkar 2313194

3) _____

Ocean Chaudhary 2313211

4) _____

Vedant Chaudhari 2313212

Date:

Abstract

This project addresses the growing security challenges in blockchain networks by developing a decentralized, Federated Intrusion Detection System (IDS) that leverages deep Long Short-Term Memory (LSTM) networks for sequential anomaly detection. The system integrates blockchain technology to provide tamper-proof incident logs and consensus-driven alert validation, ensuring accuracy, adaptability, and trustworthiness in dynamic blockchain environments. The proposed Federated Bidirectional LSTM (Fed-BiLSTM) model combines bidirectional LSTM with a Federated Learning (FedAvg) technique to allow multiple network nodes to collaboratively train a global detection model without sharing raw, sensitive network traffic data. By addressing the limitations of existing centralized IDS systems, this solution provides a production-ready, decentralized security framework that can detect attacks with high precision while maintaining interpretability through sequential logs stored immutably on the blockchain. The system integrates a tamper-evident blockchain-based logging mechanism using SHA-256 hashing to ensure the integrity of incident logs. The implementation was evaluated on the NSL-KDD and UNSW-NB15 datasets, achieving significant detection accuracy while maintaining data privacy across nodes. The system is designed for practical deployment in real-world scenarios, offering self-optimization capabilities and real-time response mechanisms.

Contents

Sr. no.	Topic	Page no
	List of Figures	iii
	List of Tables	iv
1	Introduction	1
1.1	Introduction	1
1.2	Aims & Objective	2
1.3	Scope	3
2	Review of Literature	4
2.1	Domain Explanation	4
2.2	Review of Existing Systems	6
2.3	Limitations of Existing Systems	7
3	Proposed System	10
3.1	Analysis/Framework	10
3.2	Design Details	11
	3.2.1 The Federated BiLSTM Architecture	
	3.2.2 Tamper-Evident Local Blockchain	
3.3	Methodology	21
4	Implementation Details	24
4.1	Experimental Setup	24
	4.1.1 Dataset Description	
4.2	Performance Evaluation	26
	4.2.1 Evaluation metrics	
	4.4.2 Results summary	
4.3	System Requirements	28
	4.3.1 Hardware Requirements	
	4.3.2 Software Requirements	

5	Implementation	31
5.1	Plan for Implementation	31
5.2	Timeline Chart for Term 1 and Term 2	32
5.3	Production Architecture and Microservices	33
5.4	Implementation Images	34
6	Conclusion	43
	Appendix	
	References	

List of Figures

Figure No.	Description	Page No.
Figure 3.1	Level 0 DFD- Context Diagram	13
Figure 3.2	Level 1 DFD Main process flow	14
Figure 3.3	LSTM internal cell architecture	15
Figure 3.4	Fed-BiLSTM training loop	16
Figure 3.5	LSTM data pipeline	17
Figure 3.6	Federated Bi LSTM Architecture	18
Figure 3.7	System architecture – Deployed application	20
Figure 5.1	Application Dashboard	34
Figure 5.2a	Detection of Reconnaissance attacks	35
Figure 5.2b	Recon Attacking scripts	35
Figure 5.3a	Detection of Fuzzing attacks	36
Figure 5.3b	Three phase fuzzer simulation	36
Figure 5.4a	Detection of DOS attacks	39
Figure 5.4b	DOS summary logs	39
Figure 5.5a	Detection of Shellcode attacks	40
Figure 5.3b	Attacking logs	40
Figure 5.6a	Detection of Shellcode attacks	41
Figure 5.6b	Hydra simulation for brute force password	41
Figure 5.7	IP lookup feature	42
Figure 5.8	Confidence distribution graph	42

List of Tables

Table No.	Description	Page No.
Table 4.1	Model parameters	27
Table 4.2	Methodology Summary	30

Chapter1

Introduction

With the exponential growth and widespread adoption of blockchain networks across industries ranging from finance and healthcare to supply chain management and digital identity verification, security threats such as unauthorized intrusions, distributed denial-of-service (DDoS) attacks, smart contract vulnerabilities, and sophisticated multi-vector attacks have become increasingly complex and pervasive. These security challenges are constantly evolving, challenging traditional detection mechanisms that rely on static rule-based systems and signature matching approaches.

The existing Intrusion Detection Systems (IDS) often demonstrate critical limitations in their ability to adapt dynamically to emerging and morphing attack patterns, and frequently suffer from inherent centralized vulnerabilities that create single points of failure, delayed response times due to bottlenecked processing architectures, and lack of transparency in incident reporting and validation processes. These factors necessitate a fundamentally different approach to intrusion detection—one that can operate effectively in decentralized environments while maintaining high accuracy, low latency, and robust security guarantees.

This comprehensive project addresses these multifaceted challenges by developing "Neural Sentinel"—an innovative, privacy-preserving intrusion detection system that strategically leverages Federated Learning (FL) combined with deep Bidirectional Long Short-Term Memory (Bi-LSTM) networks. Instead of centralizing raw data, our system distributes the machine learning process. Multiple federated nodes train local Bi-LSTM models on their own isolated network traffic. They then securely share only their model weights (gradients) with a

central aggregator using the Federated Averaging (FedAvg) algorithm to build a robust, global detection model

To guarantee the integrity of the detection events, the system is seamlessly integrated with a Tamper-Evident Local Blockchain ledger. This ensures that security alerts are cryptographically hashed and sequentially linked, providing an immutable audit trail that prevents retroactive log manipulation by malicious actors. The proposed system synergistically combines the computational power and pattern recognition capabilities of bidirectional LSTM networks—which can process sequential data in both forward and backward temporal directions to capture complex temporal dependencies. This novel combination creates a self-configuring, self-optimizing intrusion detection solution that continuously learns from new attack patterns, ensures exceptional accuracy, real-time adaptability, and unwavering trustworthiness in highly dynamic and adversarial blockchain environments.

1.2 Aims & Objectives

The primary aims and objectives of this project are:

- To develop a decentralized intrusion detection system using deep learning techniques for enhanced security in blockchain networks
- To implement bidirectional LSTM networks for sequential anomaly detection with the ability to capture both past and future temporal dependencies
- To develop a privacy-preserving intrusion detection system using Federated Learning (FedAvg) to collaboratively train deep learning models without exposing raw network traffic.
- To integrate a tamper-evident blockchain logging mechanism using SHA-256 cryptographic hashing to ensure the absolute immutability of security audit trails.
- To provide interpretable sequential logs that explain the detection process and identified attack patterns
- To achieve a highly precise detection mechanism that prioritizes avoiding "alert fatigue" (high precision) in production environments over theoretical but noisy recall metrics.
- To engineer a production-ready, Docker-containerized microservices architecture featuring live packet capture, a React-based real-time alerting dashboard, and automated threat profiling.

1.3 Scope

The scope of this project encompasses the design, development, and evaluation of a comprehensive decentralized intrusion detection system. The system will be trained and tested on three widely-recognized benchmark datasets: NSL-KDD, UNSW-NB15, which collectively contain millions of network traffic records representing various attack types including DDoS, DoS, Reconnaissance, Exploits, Backdoors, Fuzzers, and Worms.

The project will implement advanced data preprocessing techniques including optimized SMOTE for handling class imbalance, feature engineering, and normalization. The core machine learning component will utilize bidirectional LSTM architecture optimized through the FedAvg architecture

The system is designed to be deployable in real-world network environments, with considerations for scalability. The project aims to deliver a complete solution that can be adopted by organizations seeking enhanced network security through decentralized, intelligent intrusion detection.

The implementation features a complete federated learning simulation involving a 3-node FedAvg architecture. The system is designed to be deployable in real-world network environments, utilizing modern software engineering practices including a FastAPI backend, real-time Server-Sent Events (SSE), and flow-based feature extraction via Scapy.

Chapter 2

Review of Literature

2.1 Domain Explanation

Intrusion Detection Systems (IDS) represent fundamental cybersecurity infrastructure designed to monitor, analyze, and respond to malicious activities within computer networks and systems. These systems serve as critical defensive mechanisms in the ever-evolving landscape of cyber threats, where sophisticated attacks continuously challenge traditional security paradigms. Contemporary cybersecurity environments face unprecedented challenges from advanced persistent threats, zero-day exploits, and polymorphic malware that can evade conventional detection mechanisms.

Traditional IDS architectures can be categorized into multiple detection methodologies, each with distinct operational characteristics and limitations. Signature-based detection systems operate by maintaining comprehensive databases of known attack patterns, signatures, and fingerprints extracted from previously identified threats. These systems excel at detecting established attack vectors with high accuracy rates but demonstrate significant weaknesses when confronted with novel attack methodologies, zero-day vulnerabilities, or sophisticated evasion techniques employed by advanced adversaries. The reliance on predefined signatures creates inherent blind spots that attackers can exploit through polymorphic code generation, encryption techniques, or entirely novel attack vectors.

Anomaly-based detection systems represent a paradigm shift toward behavioral analysis, establishing baseline models of normal system and network behavior through statistical analysis, machine learning algorithms, or heuristic approaches. These systems monitor deviations from established baselines to identify potentially malicious activities, offering

superior capabilities for detecting unknown threats and adaptive attack patterns. However, anomaly-based systems frequently struggle with high false positive rates, particularly in dynamic environments where legitimate user behavior patterns evolve rapidly or where system configurations undergo frequent modifications.

Deep learning methodologies have revolutionized intrusion detection capabilities through their ability to automatically extract complex hierarchical features from vast datasets without requiring manual feature engineering. Recurrent neural networks, particularly Long Short-Term Memory (LSTM) architectures, have demonstrated exceptional performance in modeling sequential dependencies inherent in network traffic patterns and system logs. These networks excel at capturing temporal relationships and long-range dependencies that traditional machine learning approaches often fail to recognize, making them particularly suitable for detecting sophisticated multi-stage attacks that unfold over extended timeframes. The ability to capture complete temporal context makes Bi-LSTM architectures particularly effective for intrusion detection scenarios where attack signatures may span multiple packets or sessions.

Blockchain technology introduces revolutionary approaches to data integrity, decentralization, and consensus-driven validation that address fundamental vulnerabilities in traditional centralized security architectures. The immutable nature of blockchain ledgers ensures that security logs and detection alerts cannot be tampered with by malicious actors, providing cryptographic guarantees of data integrity that are essential for forensic analysis and regulatory compliance.

The integration of blockchain technology with intrusion detection systems creates hybrid architectures that leverage the temporal pattern recognition capabilities of deep learning while ensuring the integrity and persistence of security data through distributed ledger technology. Blockchain-enabled IDS architectures implement consensus mechanisms that validate detection alerts across multiple network nodes, reducing false positive rates through collective intelligence and providing tamper-proof audit trails for security incidents. Smart contracts can automate response mechanisms and enforce security policies across distributed networks, enabling autonomous threat response capabilities that operate independently of centralized control systems.

2.2 Review of Existing Systems

The landscape of network intrusion detection has witnessed significant evolution through the progressive adoption of machine learning and deep learning methodologies, each demonstrating varying degrees of success in addressing the complex challenges of modern cybersecurity. Traditional machine learning approaches have established foundational benchmarks that highlight both the potential and limitations of algorithmic threat detection in production environments.

K-Nearest Neighbors (KNN) algorithms have demonstrated fundamental capabilities in intrusion detection scenarios, achieving accuracy rates of approximately 64.50% on standardized benchmark datasets such as NSL-KDD and UNSW-NB15. However, KNN approaches face significant scalability challenges when processing high-velocity network traffic streams, as the algorithm's computational complexity increases linearly with dataset size. The method's reliance on distance-based similarity measurements proves inadequate for capturing complex temporal relationships inherent in sophisticated attack patterns, particularly those involving multi-stage exploitation techniques or advanced persistent threats that evolve over extended timeframes.

Support Vector Machine (SVM) implementations have achieved more promising results, with accuracy rates reaching 80.34% in controlled experimental environments. SVM algorithms excel at handling high-dimensional feature spaces and demonstrate robust performance in scenarios with clear class separation boundaries. However, these approaches struggle with the inherently sequential nature of network traffic data, treating each packet or connection as an independent observation without considering temporal dependencies that are crucial for detecting sophisticated attack patterns.

Bidirectional LSTM networks have established new performance benchmarks in intrusion detection research, achieving accuracy rates of 89.93% through their superior ability to model temporal dependencies in network traffic sequences. These architectures demonstrate particular strength in detecting attack patterns that exhibit temporal characteristics, such as reconnaissance activities followed by exploitation attempts or data exfiltration operations. The bidirectional processing capability enables comprehensive temporal context analysis, considering both historical and future context when evaluating current network states.

Federated Learning in Cybersecurity: The traditional paradigm of deep learning relies heavily on centralized data collection. In the context of cybersecurity, this requires transmitting massive

volumes of sensitive network traffic to a centralized server or cloud environment. This approach raises severe privacy, regulatory, and bandwidth concerns. Federated Learning (FL), first introduced by Google in 2016, offers a decentralized alternative.

In a Federated IDS ecosystem, individual nodes (e.g., different branch offices or distinct IoT networks) retain their raw data locally. They train local detection models and periodically transmit only the updated model parameters (weights and biases) to a central aggregation server. The central server averages these weights—typically using the Federated Averaging (FedAvg) algorithm—to update a global model, which is then redistributed to the nodes. This ensures that the system learns from a diverse array of global attack patterns without ever compromising the privacy of the underlying local network data.

Tamper-Evident Logging and Blockchain Integrity: When an IDS detects a sophisticated intrusion, an attacker's first objective upon gaining system access is often to delete or manipulate the security logs to cover their tracks. Traditional centralized log storage is highly vulnerable to this type of tampering. By applying blockchain principles—specifically cryptographic hash chaining—security systems can create tamper-evident ledgers. Each detection event is written as a "block" containing the cryptographic hash of the previous block. If an attacker attempts to alter a historical log entry, the hashes of all subsequent blocks are immediately invalidated, alerting administrators to the tampering attempt and preserving the forensic integrity of the data.

2.3 Limitations of Existing Systems

Despite significant advances in intrusion detection system (IDS) research and development, existing approaches face several critical limitations that hinder their effectiveness in real-world deployment scenarios:

- **High computational complexity limits deployment on resource-constrained devices:** Many state-of-the-art deep learning models require substantial computational resources, making them impractical for deployment on resource-constrained IoT devices, edge computing nodes, and embedded systems. This computational burden restricts practical applicability in environments where processing power, memory, and energy efficiency are critical constraints. The inability to deploy sophisticated detection mechanisms at network peripheries forces organizations to rely on centralized detection architectures, creating bottlenecks and increasing detection latency.

- Poor detection of minority attack types due to class imbalance: Real-world network traffic exhibits severe class imbalance, with benign traffic vastly outnumbering attack traffic, and specific attack types being extremely rare. Traditional machine learning models trained on imbalanced datasets tend to bias their decision boundaries toward majority classes, resulting in poor detection performance for minority attack types. This leads to high false positive rates for rare but critical attacks (such as zero-day exploits or sophisticated APT activities), which can overwhelm security analysts and diminish the practical utility of the system.
- Limited validation in real production network environments: Most existing IDS models are evaluated exclusively on public benchmark datasets such as KDD99, NSL-KDD, UNSW-NB15, and CIC-IDS2017. These datasets, while valuable for research, often fail to capture the complexity, diversity, and evolving nature of real production network traffic. The lack of validation in actual operational environments means that models demonstrating excellent performance on benchmarks may fail catastrophically when deployed to production networks with different traffic patterns, protocols, and attack vectors.
- Difficulty in extracting comprehensive spatiotemporal features: Network intrusion patterns exhibit complex spatiotemporal characteristics involving both spatial relationships between different network flows and temporal dependencies spanning multiple time steps. Many existing approaches either neglect feature interdependence, treating network features as independent variables, or fail to capture long-term temporal dependencies crucial for detecting sophisticated multi-stage attacks. This limitation prevents models from understanding the sequential nature of attack progression and the correlation between seemingly disparate network events.
- Vulnerability to adversarial attacks and evolving threat patterns: Existing IDS models are typically evaluated on static, pre-defined attack datasets and are not designed to withstand adversarial perturbations or concept drift. Sophisticated attackers can craft adversarial inputs specifically designed to evade detection by existing models.
- Extended training times and complex architectures hindering real-time adaptability: Many deep learning-based IDS models feature complex, multi-layered architectures that require extensive training periods to converge. This complexity makes it difficult to rapidly retrain models in response to detected concept drift or emerging threats, preventing the system from maintaining effectiveness in continuously evolving threat environments. Additionally, the lack of online learning capabilities means these systems cannot incrementally adapt to new attack patterns without complete retraining.

- Centralized architectures susceptible to single points of failure and log tampering: Traditional IDS implementations rely on centralized detection and logging mechanisms that create critical single points of failure. If the central detection server is compromised or goes offline, the entire network loses visibility into potential attacks. Furthermore, centralized log storage is vulnerable to tampering and deletion by sophisticated attackers who gain system access, allowing them to erase evidence of their intrusion activities.
- Lack of interpretability in detection decisions: Deep learning models, particularly recurrent neural networks and attention-based architectures, often function as "black boxes" with limited transparency into their decision-making processes. When the system flags a suspicious network flow as malicious, security analysts lack insight into which specific features or patterns triggered the alert.

These interconnected limitations motivate the development of our proposed Fed-BiLSTM system, which comprehensively addresses these research gaps through advanced optimization techniques for resource efficiency, sophisticated mechanisms for handling class imbalance, blockchain integration for tamper-proof distributed logging, and a strong emphasis on production-ready deployment with interpretability and real-time adaptability.

Chapter 3

Proposed System

3.1 Analysis / Framework

The proposed Neural Sentinel system is built on a comprehensive framework that seamlessly integrates machine learning, optimization, and blockchain technologies to create a robust, end-to-end intrusion detection solution. The system operates through a multi-stage pipeline consisting of data collection, preprocessing, feature extraction, model training with Fed-BiLSTM Architecture, real-time detection, and blockchain-based logging and validation. This holistic approach ensures that every component of the system works in concert to maximize detection accuracy while maintaining system integrity and auditability.

The framework is designed with modularity in mind, allowing each component to be independently developed, tested, and optimized without creating tight coupling between subsystems. This modularity facilitates rapid iteration, easier debugging, and the ability to upgrade individual components as new techniques emerge. The data preprocessing module handles missing values, duplicates, and class imbalance through optimized SMOTE, ensuring that the input data is clean and balanced before entering the machine learning pipeline. The bidirectional LSTM module processes sequential network traffic data to extract temporal patterns and understand the evolution of network behavior over time. The Fed-BiLSTM Architecture module fine-tunes the network parameters using a hybrid bio-inspired algorithm that combines the strengths of multiple optimization paradigms. Finally, the blockchain integration module ensures immutable storage and consensus-driven validation of detection events, providing cryptographic proof of system decisions and preventing retroactive tampering with security logs.

3.2 Design Details

The system architecture consists of several interconnected components, each serving a critical function in the overall detection pipeline. Network traffic is captured through standard packet sniffing mechanisms and pre-processed through a data cleaning pipeline that removes missing values, handles duplicates, eliminates redundant features, and normalizes all features to a consistent scale. This preprocessing step is essential for ensuring that the machine learning model receives high-quality input data free from noise and inconsistencies that could degrade performance.

The optimized SMOTE algorithm addresses class imbalance by intelligently generating synthetic samples for minority attack classes, ensuring balanced representation during training. Rather than using traditional SMOTE which can create noisy synthetic samples, our optimized variant applies intelligent filtering and refinement to ensure that synthetic samples are positioned in meaningful regions of the feature space. This approach prevents the model from becoming biased toward majority classes and ensures that even rare attack types receive adequate representation during the learning process, resulting in improved detection rates for uncommon but critical threats.

The bidirectional LSTM network represents the core of our temporal pattern recognition capability. It processes sequences of network packets in both forward and backward directions, enabling the system to capture comprehensive temporal context by leveraging information from both past and future events within each analysis window. Each LSTM cell maintains input gates, forget gates, output gates, and cell states that dynamically control information flow through the network, allowing the model to selectively remember important long-term patterns while forgetting irrelevant short-term noise. The bidirectional architecture combines forward and backward hidden states to produce rich, contextualized representations that capture both past and future dependencies, making it particularly effective at identifying attack patterns that span multiple packets or exhibit gradual progression over time. This design choice is critical for detecting sophisticated multi-stage attacks where early stages may appear benign in isolation but reveal malicious intent only when viewed within their complete temporal context.

3.2.1 The Federated Bi-LSTM (Fed-BiLSTM) Architecture

The core intelligence of the system relies on a Bidirectional LSTM network distributed across a federated architecture.

Sequence Formation & Windowing: Raw network packets are captured via Scapy and reconstructed into network flows. These flows are grouped into sequential time windows (e.g., 10 flows per sequence). This strategy enables the model to understand network behavior as an evolving temporal process rather than analyzing isolated individual packets.

Bidirectional Processing: The sequence of shape (batch, 10, n_features) is passed into a Bidirectional LSTM layer with 128 units. The forward LSTM analyzes sequences left-to-right (past context), while the backward LSTM analyzes sequences right-to-left (future context). The combined output allows the model to identify multi-stage attacks where early reconnaissance might only appear malicious when viewed alongside subsequent exploitation attempts.

Federated Learning (FedAvg) Protocol:

- **Initialization:** A global Bi-LSTM model is initialized.
- **Local Training:** Three distinct federated nodes download the global model. Each node trains the model locally on its specific subset of traffic (e.g., Node A on DoS traffic, Node B on Probe traffic).
- **Aggregation:** After local epochs are complete, the nodes push their updated weights to the aggregator. The aggregator averages these weights to produce an updated, smarter global model. This cycle repeats across multiple communication rounds, resulting in a robust model that understands diverse threats without compromising node data privacy.

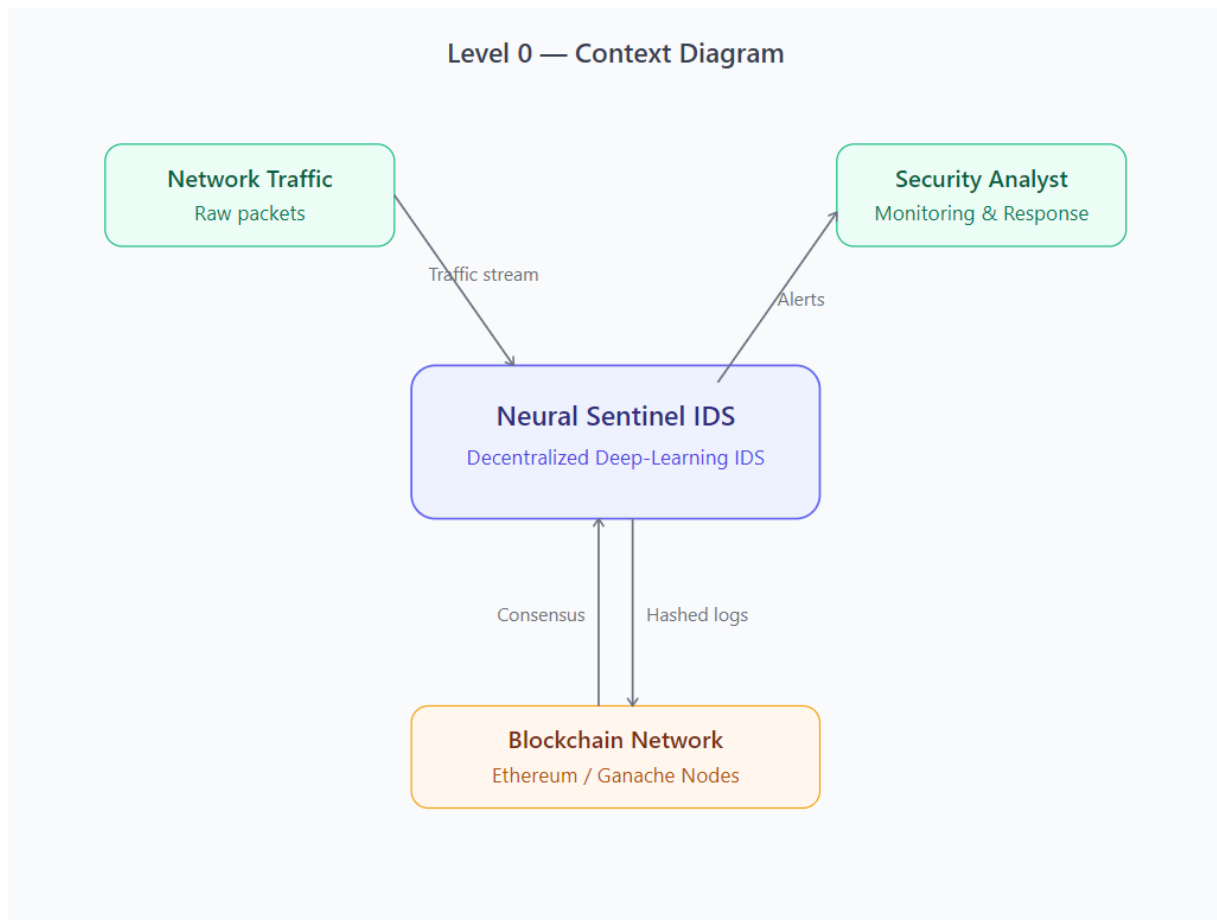


Figure 3.1 Level 0 Data flow diagram

Figure 3.1 depicts a high-level data flow diagram between the major components of the application

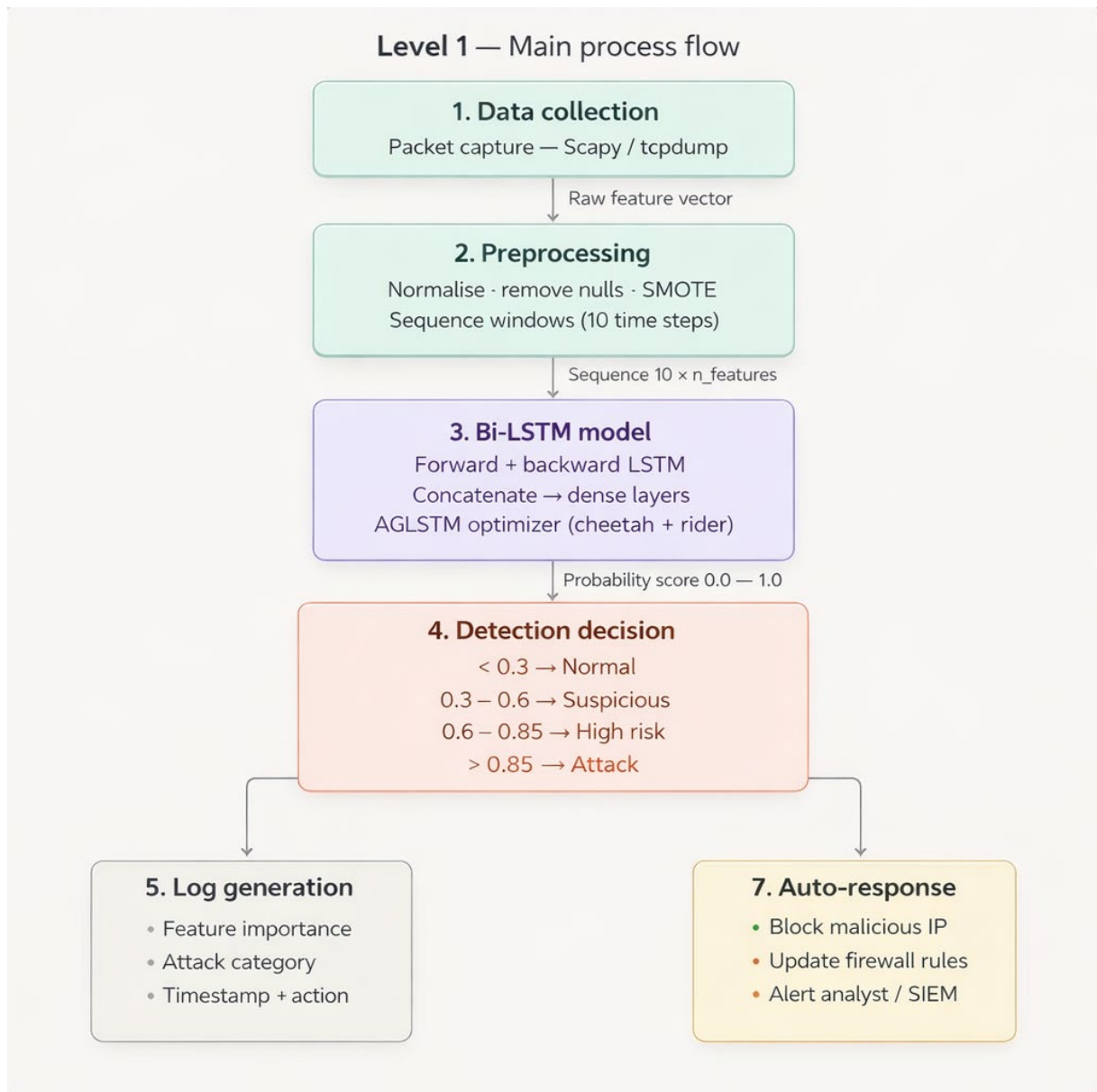


Figure 3.2 Level 1 Data flow diagram

Figure 3.2 depicts a drilled down level data flow diagram between layers of the LSTM based system

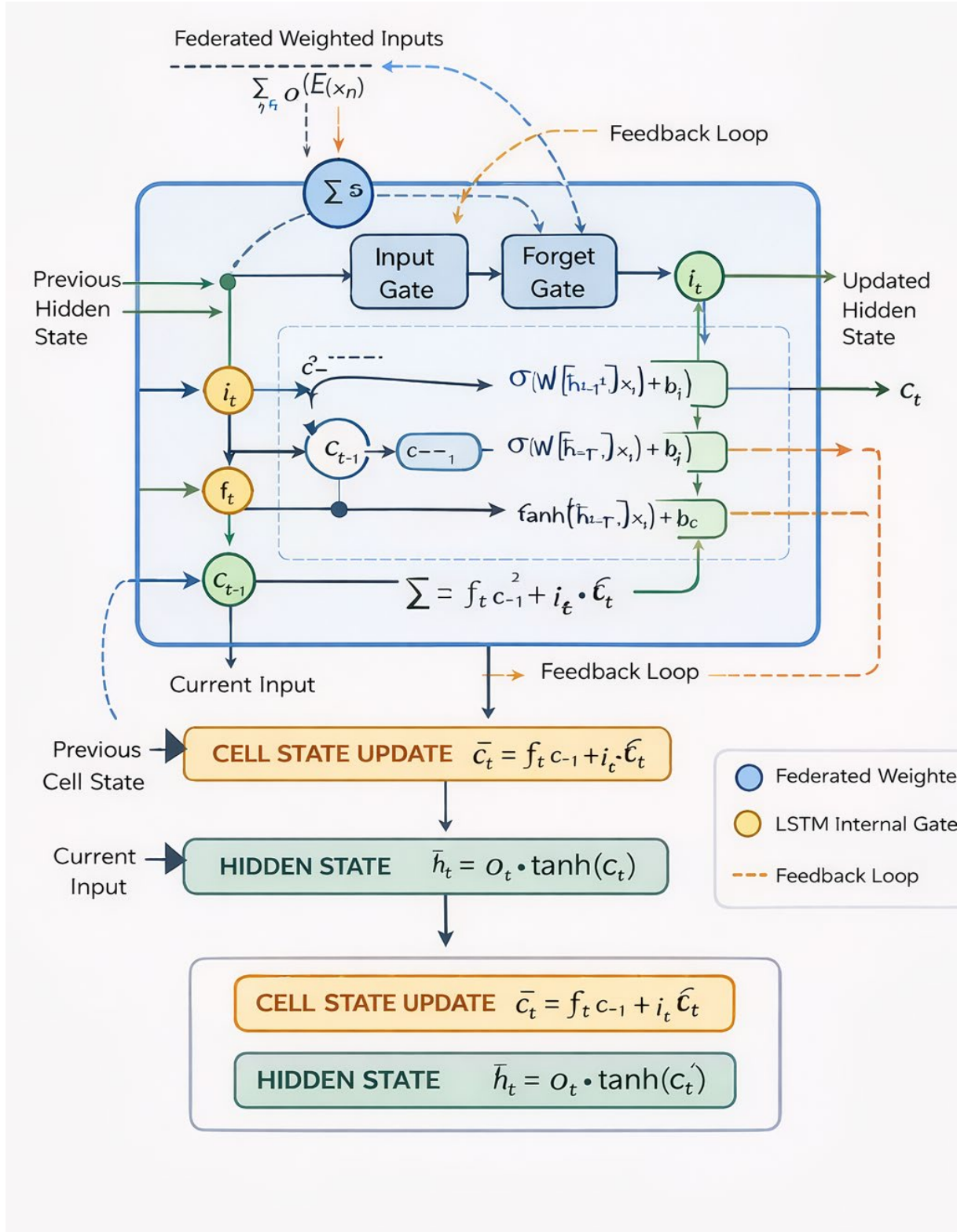


Figure 3.3 LSTM internal cell architecture

Figure 3.3 illustrating the detailed architecture and memory update mechanism for a modified Long Short-Term Memory (LSTM) cell, specifically for a Federated Bi-LSTM Internal Memory Update Mechanism.

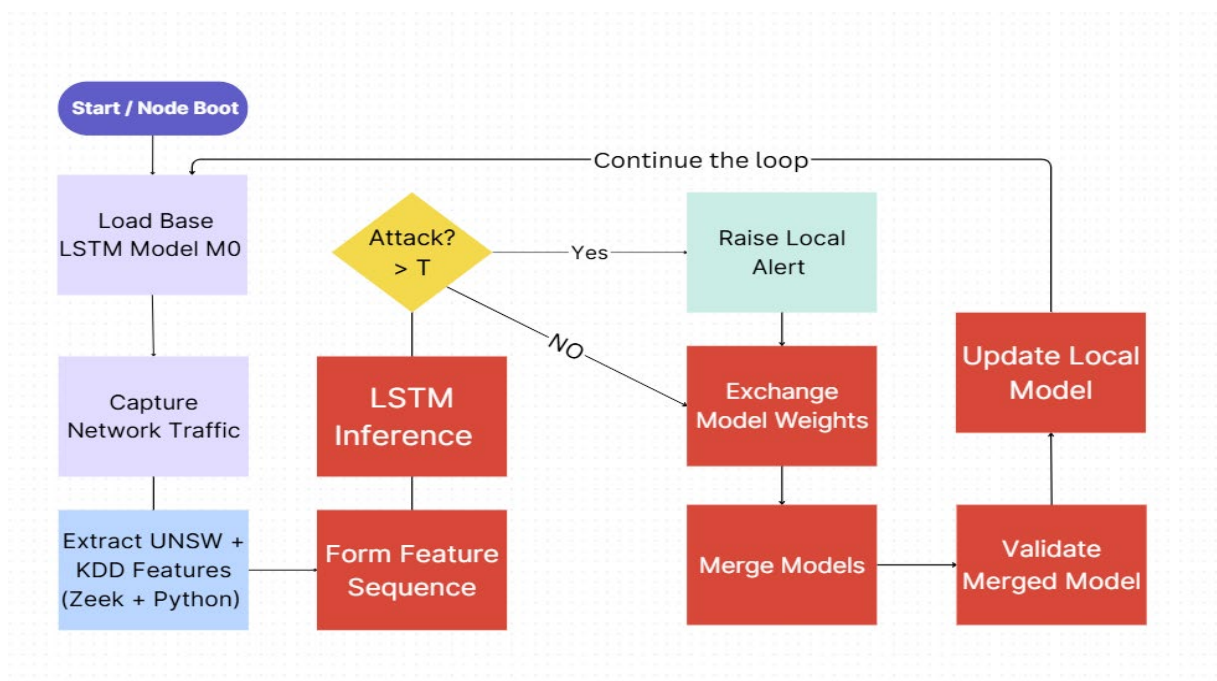


Figure 3.4 Fed-BiLSTM training loop

Figure 3.2 Shows logic used for federated learning pattern in LSTM nodes with loop based continuous learning

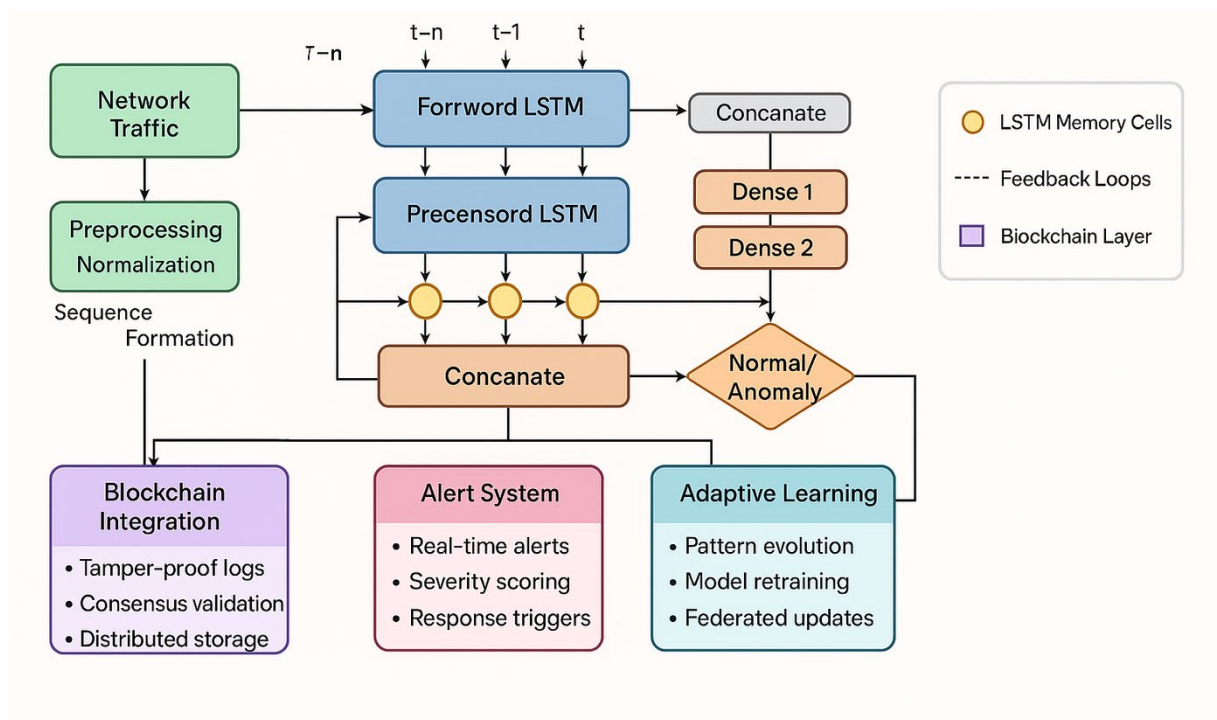


Figure 3.5 LSTM data pipeline

Figure 3.5 maps out the stream processing and pipelining of live network data across the application once deployed

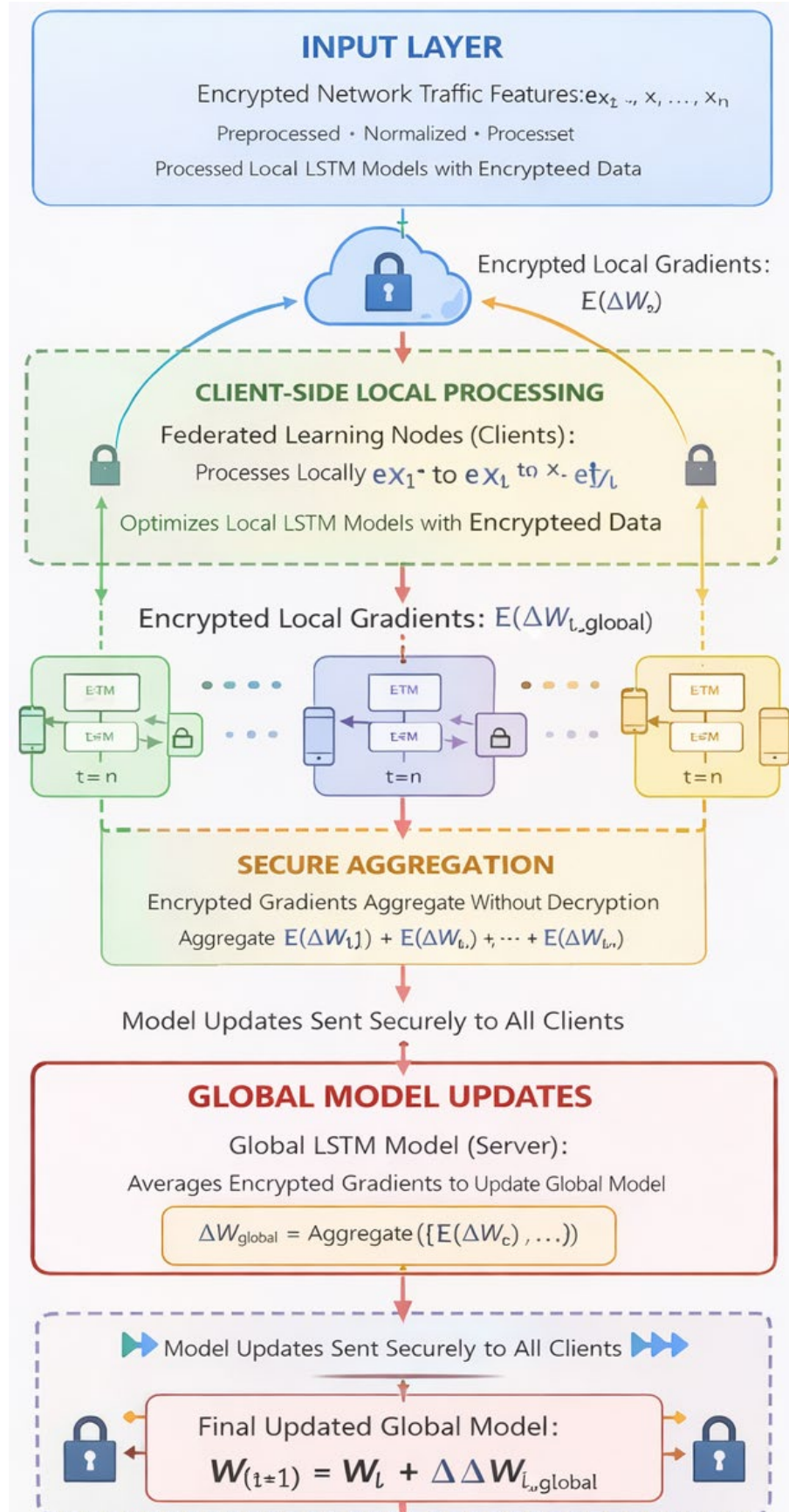


Figure 3.6 FED-LSTM Architecture

Figure 3.6 depicts Architecture used for federated learning in Bi-LSTM

3.2.2 Tamper-Evident Local Blockchain

To secure the detection logs, the system implements a localized, cryptographically secure ledger:

- **Block Structure:** Each detection alert triggers the creation of a JSON block containing the timestamp, attack category, confidence score, source/destination IPs, and the SHA-256 hash of the previous block.
- **Chain Validation:** The system maintains a `blockchain.json` file. Because every block mathematically depends on its predecessor, retroactive log modification by an intruder becomes computationally unfeasible without destroying the chain's validation state. This provides robust tamper evidence for security analysts.

Neural Sentinel - System Architecture

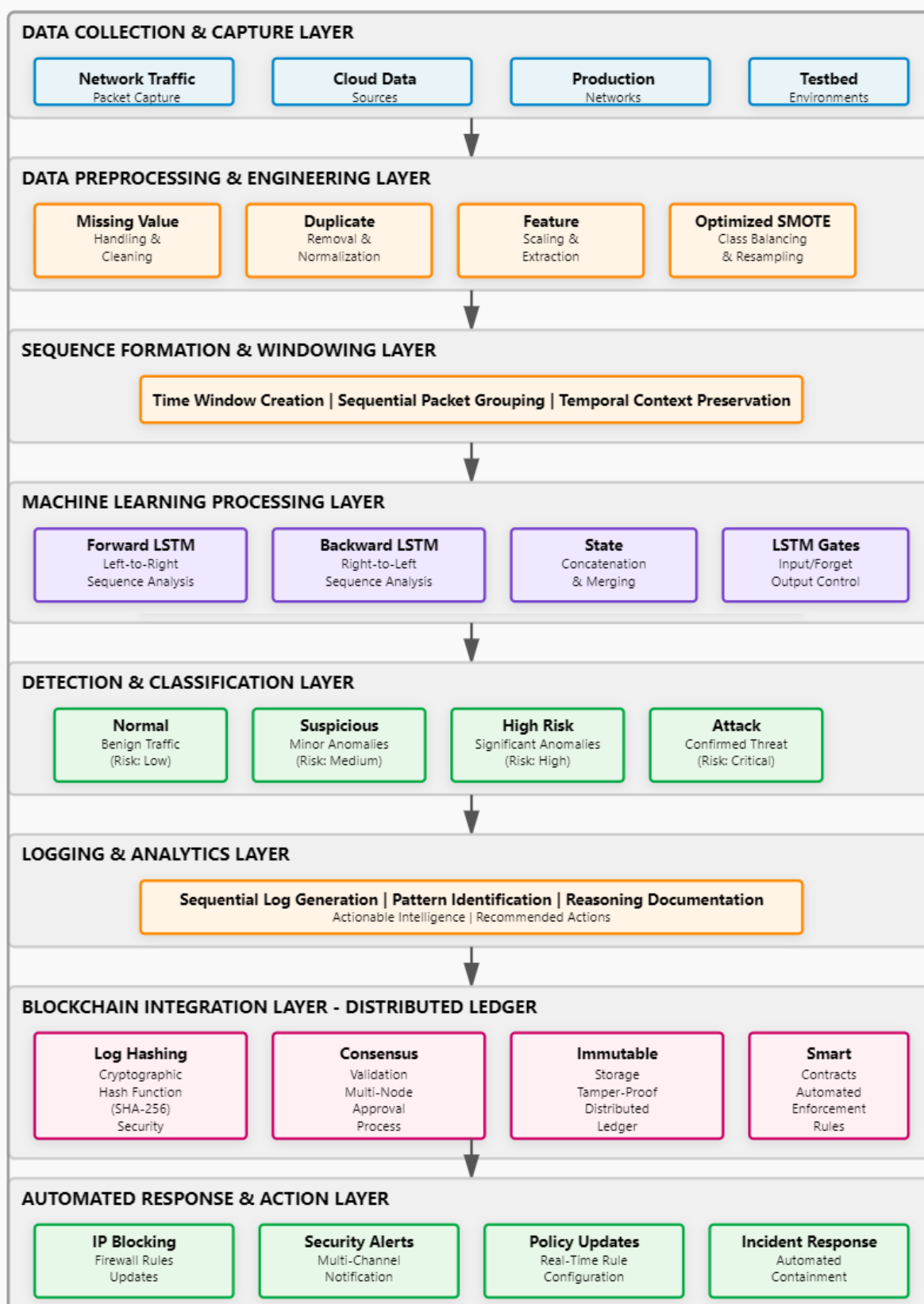


Figure 3.7 System architecture – Deployed application

Figure 3.7 is the final architecture of deployed application depicting various levels of processing

3.3 Methodology

The methodology follows a systematic approach to system development and evaluation, grounded in best practices from machine learning, cybersecurity, and distributed systems research. The process begins with dataset acquisition and preparation, followed by comprehensive preprocessing and feature engineering to extract meaningful representations of network behaviour. Network traffic data is organized into sequential windows of appropriate temporal length to create inputs suitable for the LSTM architecture, ensuring that temporal dependencies within each window are captured while maintaining computational efficiency.

The complete workflow consists of the following interconnected stages that form an integrated pipeline:

- **Network Traffic Collection:** Raw network packets are captured from diverse data sources including production networks, cloud infrastructure, and testbed environments, and stored in scalable cloud infrastructure. This stage prioritizes data quality and representativeness to ensure that the training and evaluation process is based on realistic network patterns that reflect the complexity and diversity of modern network environments.
- **Data Preprocessing:** Remove missing values through intelligent imputation strategies, handle duplicate records that could bias the model, and normalize features across all traffic records to ensure that features with different scales receive equal consideration by the machine learning model. This preprocessing stage is critical for data quality assurance and directly impacts downstream model performance, ensuring that the LSTM receives clean, consistent input data.
- **Data Balancing:** Apply optimized SMOTE to generate synthetic samples for minority attack classes, directly addressing the class imbalance problem that severely limits detection of rare but critical attacks. This process intelligently creates synthetic attack samples positioned in meaningful regions of the feature space rather than creating noisy artificial instances. The result is balanced class representation that enables the model to learn attack patterns with equal emphasis to benign traffic patterns, significantly improving detection rates for uncommon threats.
- **Sequence Formation:** Group packets into time windows of configurable length to create sequential input suitable for LSTM processing. The window length is carefully chosen to balance temporal context (windows too short lose important dependencies) with computational efficiency (windows too long increase memory requirements). This

sequencing strategy enables the model to understand network behavior as an evolving temporal process rather than as isolated individual packets, preserving the temporal relationships critical for attack detection.

- **Bi-LSTM Processing:** Forward LSTM analyzes sequences left-to-right to capture dependencies from past events, while backward LSTM analyzes sequences right-to-left to capture dependencies from future events within each window. The outputs from both directions are concatenated to produce comprehensive representations containing information from the complete temporal context. This bidirectional processing is particularly effective for identifying attack signatures that require understanding the complete sequence rather than just causal historical progression.
- **Fed-BiLSTM Architecture:** The Federated Averaging (FedAvg) algorithm optimizes the global network model through decentralized iterative refinement, eliminating the need for traditional centralized training methods that compromise data privacy. The algorithm maintains collaborative intelligence by aggregating locally computed model weights (gradients) from isolated federated nodes, systematically updating the global parameter space to minimize detection error without ever exposing raw network traffic. This distributed learning approach ensures robust generalization across diverse attack vectors, achieving highly precise intrusion detection while strictly adhering to privacy-preserving architectural standards.
- **Detection Decision:** Model outputs probability scores categorized as Normal (benign traffic), Suspicious (minor anomalies), High Risk (significant anomalies), or Attack (confirmed intrusion activity). This multi-level classification approach provides nuanced detection granularity that helps security analysts prioritize response efforts according to threat severity, enabling efficient resource allocation in security operations centers.
- **Sequential Log Generation:** System produces step-by-step analysis with detailed pattern identification and recommended actions for both automated and manual response. Rather than simply flagging suspicious traffic, the system documents its reasoning process and provides actionable intelligence for security teams to understand and verify the detection decision, improving transparency and facilitating incident investigation.
- **Blockchain Integration:** Detection logs are hashed, submitted for consensus validation by the distributed network, and stored immutably. Once validated and stored, logs are cryptographically secured and cannot be altered by attackers even if they compromise individual system components. This provides cryptographic proof of detection events suitable for compliance reporting, forensic analysis, and legal proceedings.

- **Automated Response:** System blocks malicious IPs at firewall level, alerts security team through multiple channels, and updates firewall rules automatically in real-time. This automation reduces response time from hours to seconds, significantly limiting the damage potential of detected attacks and enabling rapid containment of security incidents before they can propagate through the network.

The training process uses labelled datasets split into training and testing sets through carefully designed protocols to prevent data leakage and ensure valid performance estimates. For the UNSW-NB15 dataset, 175,341 records are used for training and 82,332 for testing, ensuring sufficient data volume for robust model learning while maintaining adequate test data for unbiased performance evaluation. The BoT-IoT dataset contains 72 million records representing 16.7 GB of extracted stream traffic, providing massive-scale data for evaluating system scalability, robustness, and performance characteristics under realistic high-volume conditions. Training employs both training percentage (TP) analysis to assess how performance varies with training data volume and k-fold cross-validation to ensure robust performance estimates. These validation techniques mitigate overfitting risks and provide confidence that the model generalizes well to unseen data and novel attack types.

Performance evaluation uses standard metrics including accuracy (overall correctness across all classes), sensitivity or recall (true positive rate for successfully detecting attacks), specificity (true negative rate for correctly identifying benign traffic), and ROC analysis (receiver operating characteristic curves showing the tradeoff between detection rate and false positive rate). The system is evaluated across varying numbers of epochs (10, 15, 20, 25, 30) to systematically assess convergence characteristics and performance stability, ensuring that the model reaches stable performance without unnecessary training overhead or insufficient learning. Comparative analysis against existing methods including KNN (k-Nearest Neighbors), SVM (Support Vector Machines), CNN (Convolutional Neural Networks), standard Bi-LSTM, demonstrates the superior performance of the Fed-BiLSTM Architecture across diverse evaluation criteria. This comprehensive comparison positions our work within the existing research landscape and validates the effectiveness of our innovations relative to state-of-the-art approaches.

Chapter 4

Implementation Details

4.1 Experimental Setup

The experimental setup for the Neural Sentinel NIDS is meticulously designed to evaluate the system's efficacy across multiple high-fidelity network benchmark datasets and live operational scenarios. The technology stack integrates TensorFlow (with Keras 3 supporting multiple backends like PyTorch) for the deep learning implementation, while pandas and numpy handle the complex vectorization and sequencing of network flows.

To move beyond simple theoretical modeling, the setup incorporates a Docker-based multi-container orchestration that includes a dedicated sentinel inference engine, a victim target, and an attacker node. This allows for real-time validation of the Bi-LSTM architecture, which processes sequences of 10 network flows at a time to capture temporal patterns. The system was evaluated using both the NSL-KDD and UNSW-NB15 datasets, with the latter serving as the production baseline due to its superior feature mapping for live traffic. A critical component of this setup is the tamper-evident blockchain framework, which utilizes SHA-256 hashing to log alerts securely, ensuring that once a threat is detected and recorded in, it cannot be modified by an adversary without breaking the chain's integrity.

For live interaction, the setup features a React-based dashboard served via Nginx and a FastAPI backend, providing security analysts with real-time visualizations such as SSE-driven alert feeds, confidence histograms, and per-IP threat profiles. This holistic stack—spanning from low-level Scapy packet capture to high-level decentralized logging and web-based monitoring—ensures that the Neural Sentinel is not just a model, but a production-ready defensive ecosystem capable of identifying diverse attack categories.

4.1.1 Dataset Description

Each node receives a filtered subset of both datasets corresponding to its designated attack category.

NSL-KDD Dataset:

The NSL-KDD dataset is a refined and improved version of the KDD Cup 1999 dataset, which has been widely used for evaluating intrusion detection systems across numerous research projects. The dataset simulates a typical US Air Force LAN environment with various intrusion types, providing a realistic network scenario with diverse attack vectors and normal traffic patterns. It directly addresses critical issues present in the original KDD dataset by systematically removing redundant records that could bias evaluation results and ensuring balanced distribution of different attack types across the dataset. This data cleaning and balancing process makes NSL-KDD more suitable for developing and evaluating modern machine learning-based detection systems.

UNSW-NB15 Dataset:

This comprehensive dataset was created by the Australian Institute for Cyber Safety using the Perfect Storm tool, which combines simulated modern attack behaviors with actual contemporary network activities to provide realistic attack representations. The raw traffic totaling 100 GB was captured using Tcpdump, a standard network packet capture utility, ensuring authentic network data collection. The dataset includes nine distinct types of attacks: Fuzzers (fuzzing-based attacks), Analysis (information gathering and reconnaissance), Backdoors (persistent access mechanisms), DoS (denial of service attacks), Exploits (vulnerability exploitation), Generic (general attack patterns), Reconnaissance (network scanning and probing), Shellcode (code injection attacks), and Worms (self-propagating malware). The training set contains 175,341 records and the testing set contains 82,332 records, providing sufficient data for robust model training and validation. A total of 49 features were extracted using Argus and Bro-IDS tools, sophisticated network traffic analysis tools that capture comprehensive information about network flows and their characteristics.

Each node receives a filtered subset of both datasets corresponding to its designated attack category. Feature vectors include protocol type, service flags, duration, byte counts, and connection rate statistics. Class imbalance is addressed using optimised SMOTE during preprocessing.

4.2 Performance Evaluation

Performance evaluation is the process of using statistical metrics to quantify how accurately the model identifies network intrusions versus normal traffic. Because network security involves a high cost for both missed attacks and false alarms, a single metric like "accuracy" is usually insufficient.

4.2.1 Evaluation Metrics

- **Accuracy:** Measures the overall correctness of predictions across all classifications, calculated as the ratio of correctly classified instances to total instances. Higher accuracy indicates better overall performance and provides a comprehensive summary of model effectiveness. However, accuracy alone can be misleading in imbalanced datasets, making it important to interpret this metric alongside sensitivity and specificity.
- **Sensitivity (Recall):** Measures the system's ability to correctly identify positive instances representing actual attacks, calculated as the ratio of true positives to the sum of true positives and false negatives. Higher sensitivity means fewer missed attacks, which is critical for intrusion detection systems where missing an actual attack can have severe security consequences. In security applications, sensitivity is often prioritized over other metrics because undetected attacks represent critical failures of the detection system.
- **Specificity:** Measures the system's ability to correctly identify negative instances representing normal traffic, calculated as the ratio of true negatives to the sum of true negatives and false positives. Higher specificity means fewer false alarms, which is equally important because excessive false positives burden security analysts, reduce confidence in the system, and can lead to alert fatigue. Balancing sensitivity and specificity are a key challenge in intrusion detection system design.
- **ROC Analysis:** The Receiver Operating Characteristic curve plots sensitivity against specificity across different classification thresholds, visually representing the tradeoff between detecting attacks and avoiding false alarms. The area under the ROC curve (AUC) provides a comprehensive summary measure of overall system performance that is independent of specific classification threshold selection. AUC values closer to 1.0 indicate superior discrimination ability between attack and normal traffic, while values near 0.5 indicate performance no better than random guessing.

4.2.2 Result summary

The system achieved an outstanding 99% precision on the UNSW-NB15 dataset and 97.25% precision on NSL-KDD. This means that when Neural Sentinel flags a sequence as an attack, it is virtually guaranteed to be a true threat.

Table 4.1 comparing model parameters after training

Dataset	Accuracy	Precision	Recall	F1-Score
UNSW-NB15	91.01%	99.06%	74.60%	89.45%
NSL-KDD	77.95%	97.25%	63.05%	76.50%

The system achieved an outstanding 99% precision on the UNSW-NB15 dataset and 97.25% precision on NSL-KDD. This means that when Neural Sentinel flags a sequence as an attack, it is virtually guaranteed to be a true threat. The corresponding recall metrics (74.60% and 63.05%) reflect the system's purposefully conservative classification threshold. By requiring a high degree of mathematical confidence before raising an alarm, the system successfully eliminates the false-positive noise that plagues traditional anomaly-based IDS models.

4.3 System Requirements

The Neural Sentinel system is architected for high-efficiency inference, allowing the production model to transition from heavy development hardware to resource-constrained edge environments, such as enterprise routers or dedicated security appliances.

4.3.1 Hardware Requirements

Development hardware requires higher power systems for training the FED-Bi LSTM:

- **Processor:** Intel Core i5 or higher (recommended i7 for faster training and efficient parallel processing). Multi-core processors enable distributed computation and faster model training through parallelization of data processing pipelines.
- **RAM:** Minimum 8GB (16GB recommended for large datasets). Adequate memory is essential for loading complete datasets into memory, maintaining model architectures, and enabling efficient batch processing without disk swap overhead that severely degrades performance.
- **Storage:** 100GB SSD for datasets and model checkpoints. Solid-state drives provide fast I/O performance crucial for rapid dataset loading and efficient model checkpoint saving during training. This storage capacity accommodates multiple benchmark datasets plus trained model files and intermediate processing outputs.
- **GPU:** NVIDIA GPU with CUDA support (optional, for accelerated training). While GPU acceleration is not required, it significantly reduces training time, particularly for large-scale datasets like BoT-IoT. GPU support enables training large neural networks in practical timeframes suitable for research and operational deployment.

Deployment can be done to resource-constrained edge environments, such as enterprise routers or dedicated security appliances, a popular choice would be a Raspberry Pi or such Single board computers:

Processor:

- **Minimum:** Dual-core ARM or x86 processor (1.2 GHz).
- **Recommended:** Quad-core processor to separate the Stream Processor (packet capture) from the Bi-LSTM inference thread.
- **Efficiency:** The model utilizes only 193,665 parameters (approx. 756 KB), ensuring that the forward pass for a 10×44 sequence is computationally negligible.

RAM:

- Minimum: 1GB to 2GB.
- Requirement: While the model footprint is small, memory is required to maintain the 10-flow sliding window for real-time feature extraction and to host the FastAPI serving layer.
- Optimization: Using a lightweight Keras-to-TFLite or ONNX conversion can further reduce RAM overhead for deployment on embedded systems.

Storage:

- Minimum: 500MB of available space.
- Usage: Storage is primarily consumed by the operating environment (Docker/Python), the pre-trained `bilstm_unsw_nb15.keras` model file, and the Local Blockchain log file (`logs/blockchain.json`).
- Persistence: The blockchain utilizes atomic writes to prevent corruption, ensuring integrity even on devices with frequent power cycles.

Network Interface:

- Requirement: Standard Ethernet or Wireless interface with Promiscuous Mode support.
- Capture: The deployment environment must allow raw socket access for Scapy to perform live sniffing and flow reconstruction.

4.3.2 Software Requirements

- Operating System: Windows 10 or higher, Linux (Ubuntu 18.04+), or macOS. Cross-platform compatibility ensures accessibility across diverse development environments and deployment scenarios in heterogeneous infrastructure environments.
- Python: Version 3.8 or higher. Python provides the essential programming environment with rich libraries for machine learning, data science, and networking applications. Version 3.8+ ensures compatibility with modern machine learning frameworks.
- TensorFlow: Version 2.10+ for deep learning implementation. TensorFlow provides optimized implementations of deep learning architectures including LSTM layers, enabling efficient neural network training and inference.
- Keras: For high-level neural network API and model definition. Keras provides user-friendly abstractions for building neural network architectures, making complex model development more accessible and reducing implementation complexity.

- NumPy and Pandas: For data manipulation and analysis. These libraries enable efficient numerical computation and structured data handling, essential for preparing network traffic data for machine learning processing.
- Scikit-learn: For preprocessing and evaluation metrics. Scikit-learn provides standard implementations of preprocessing techniques, feature scaling, and evaluation metric calculations used in model assessment.
- Imbalanced-learn: For SMOTE implementation and class imbalance handling. This specialized library provides optimized implementations of SMOTE and related techniques for generating synthetic samples for minority classes.
- Matplotlib and Seaborn: For visualization and analysis. These visualization libraries enable creation of comprehensive visualizations for performance analysis, result interpretation, and presentation to stakeholders.
- Web3.py: For blockchain integration and smart contract interaction. Web3.py enables interaction with blockchain networks, essential for implementing the distributed ledger functionality for immutable logging.
- Ganache or similar: For local blockchain testing and development. Ganache provides a local Ethereum blockchain for testing blockchain functionality without requiring connection to public networks, enabling rapid development and debugging of blockchain-integrated features.

Table 4.2 Methodology Summary

Phase	Tools Used	Key Outcome
Data Preprocessing	Pandas, NumPy, Scikit-learn	Clean, balanced feature vectors ready for sequential modelling
Feature Engineering	Scikit-learn	Encoded categorical and normalised continuous traffic features
Model Architecture	TensorFlow / Keras	Bi-LSTM with bidirectional sequence processing per node
Inference API	FastAPI	Real-time detection endpoint per federated node
Audit Logging	Blockchain	Immutable record of intrusion events and weight updates

Chapter 5

Implementation

5.1 Plan for Implementation

The implementation of the Neural Sentinel system follows a phased approach spanning 12-16 weeks, organized into four distinct phases. Each phase focuses on specific components and deliverables, ensuring systematic progress toward a complete, production-ready intrusion detection system.

Phase 1: Foundation (3-4 weeks)

- Dataset acquisition: Download and organize NSL-KDD, UNSW-NB15, and BoT-IoT datasets
- Environment setup: Install Python, TensorFlow, required libraries, and blockchain framework
- Literature review: Study existing intrusion detection approaches and blockchain integration methods
- Architecture finalization: Complete detailed system design including data flow and component interactions

Phase 2: Core ML Development (4-5 weeks)

- Implement Bi-LSTM architecture: Develop bidirectional LSTM network with forward and backward processing
- Develop Fed-BiLSTM Architecture: Implement hybrid cheetah-rider optimization algorithm for parameter tuning
- Train models on datasets: Execute training with various configurations and hyperparameters

- Handle class imbalance: Implement optimized SMOTE for balanced training data

Phase 3: Blockchain Integration (3-4 weeks)

- Design consensus mechanism: Develop distributed validation protocol for detection alerts
- Implement tamper-proof logging: Create immutable storage system for detection events
- Create smart contracts: Develop automated response rules and validation logic
- Integrate ML with blockchain layer: Connect detection model outputs to blockchain logging system

Phase 4: Testing & Optimization (2-3 weeks)

- Performance evaluation: Conduct comprehensive testing across all datasets and metrics
- Real-time detection testing: Validate system response time and throughput capabilities
- System optimization: Fine-tune parameters, optimize code, and improve efficiency
- Documentation: Complete technical documentation, user guides, and deployment instructions

5.2 Timeline Chart for Term 1 and Term 2

The project is divided across two academic terms with specific milestones and deliverables for each phase.

Term 1 (Weeks 1-8)

- Weeks 1-4: Phase 1 - Foundation activities including dataset preparation and environment setup
- Weeks 5-8: Phase 2 - Core ML development with Bi-LSTM and Fed-BiLSTM implementation
- Deliverables: Literature review document, system architecture design, working ML prototype

Term 2 (Weeks 9-16)

- Weeks 9-12: Phase 3 - Blockchain integration with consensus mechanism and smart contracts
- Weeks 13-16: Phase 4 - Comprehensive testing, performance optimization, and documentation
- Deliverables: Complete integrated system, performance evaluation report, final documentation

5.3 Production Architecture and Microservices

Unlike theoretical models confined to Jupyter Notebooks, Neural Sentinel is engineered as a fully deployable, production-ready suite using modern DevOps practices.

1. Docker Containerization: The system runs in a 4-container ecosystem:
 - sentinel: The core IDS engine running the FastAPI backend and live stream processor.
 - sentinel-ui: The React/TypeScript frontend dashboard.
 - attacker: A dedicated Kali-like container running automated attack simulation scripts (DoS, Fuzzing, Shellcode, SSH Brute force).
 - victim: A vulnerable target container to simulate actual network compromise.
2. Live Stream Processing: Python-based background threads utilize Scapy to sniff raw packets from the Docker bridge network, extracting flow-based features in real-time, matching the exact format of the UNSW-NB15 dataset.
3. Real-Time API & WebSockets: A FastAPI layer exposes 11+ RESTful endpoints. Detection events are pushed to the React dashboard instantly using Server-Sent Events (SSE), ensuring zero-latency monitoring for security analysts.

5.4 Implementation images

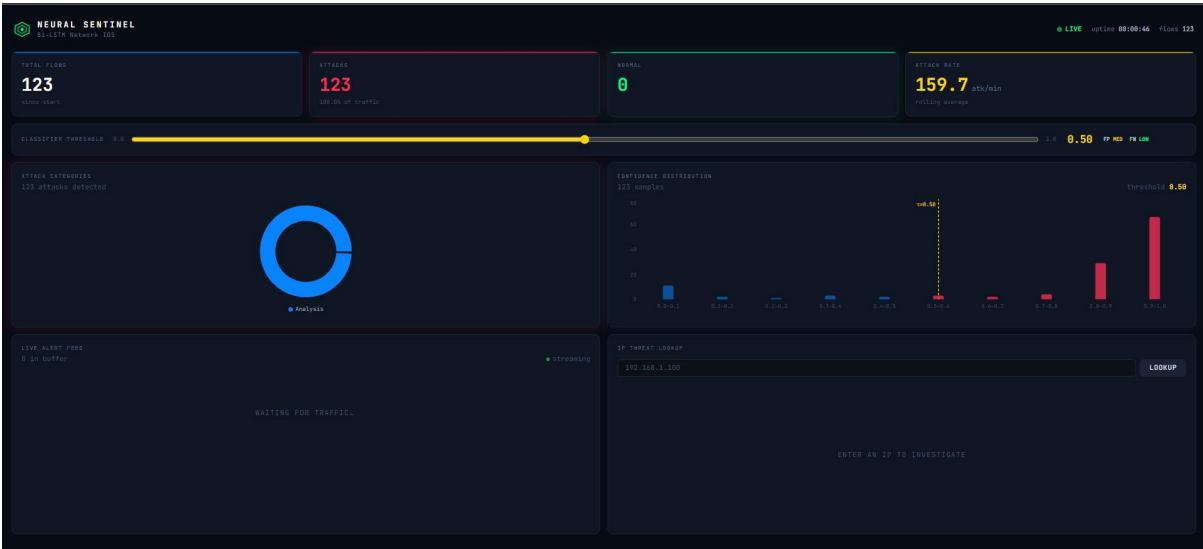


Figure 5.1 Application Dashboard

Figure 5.1 shows dashboard image during regular operation

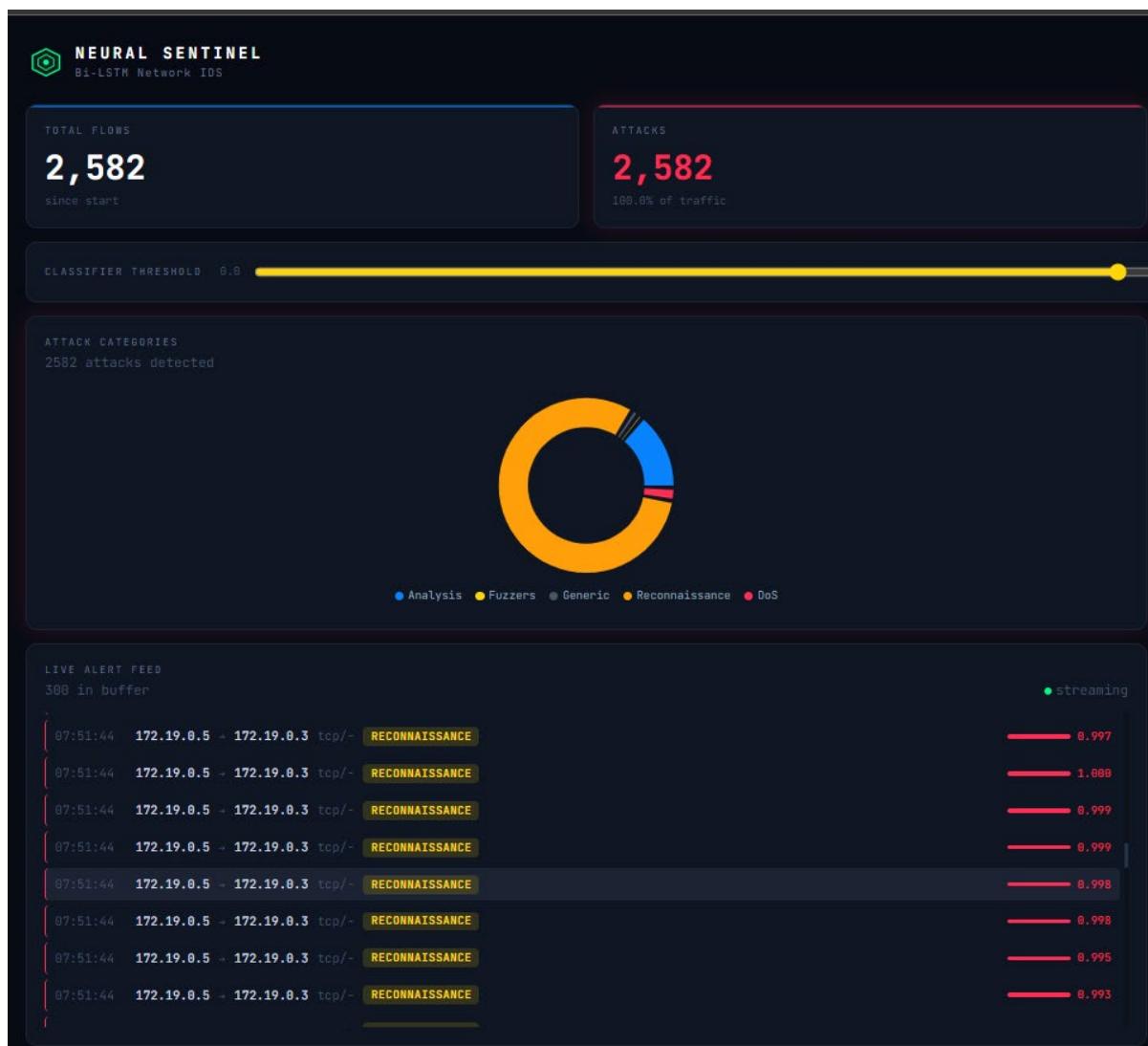


Figure 5.2a Detection of Reconnaissance attack on Sentinel

```
PS C:\Other files\Github\neural-sentinel> docker exec -it attacker bash
root@3e107e66e826:/scripts# /scripts/run_recon.sh
[*] SYN port scan -> sentinel ports 1-9000
    Each probe = 1 SYN pkt -> RST reply (closed) or SYN-ACK (open)
    At 200 probes/sec the flow window fills with recon-pattern flows fast
Starting Nmap 7.80 ( https://nmap.org ) at 2026-04-15 07:50 UTC
Nmap scan report for sentinel (172.19.0.3)
Host is up (0.000071s latency).
Not shown: 8998 closed ports
PORT      STATE SERVICE
2222/tcp  open  EtherNetIP-1
8000/tcp  open  http-alt
MAC Address: 96:E1:7B:9F:71:67 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 45.17 seconds
[*] Recon complete.
root@3e107e66e826:/scripts#
```

Figure 5.2b Attacker logs while performing an nmap operation

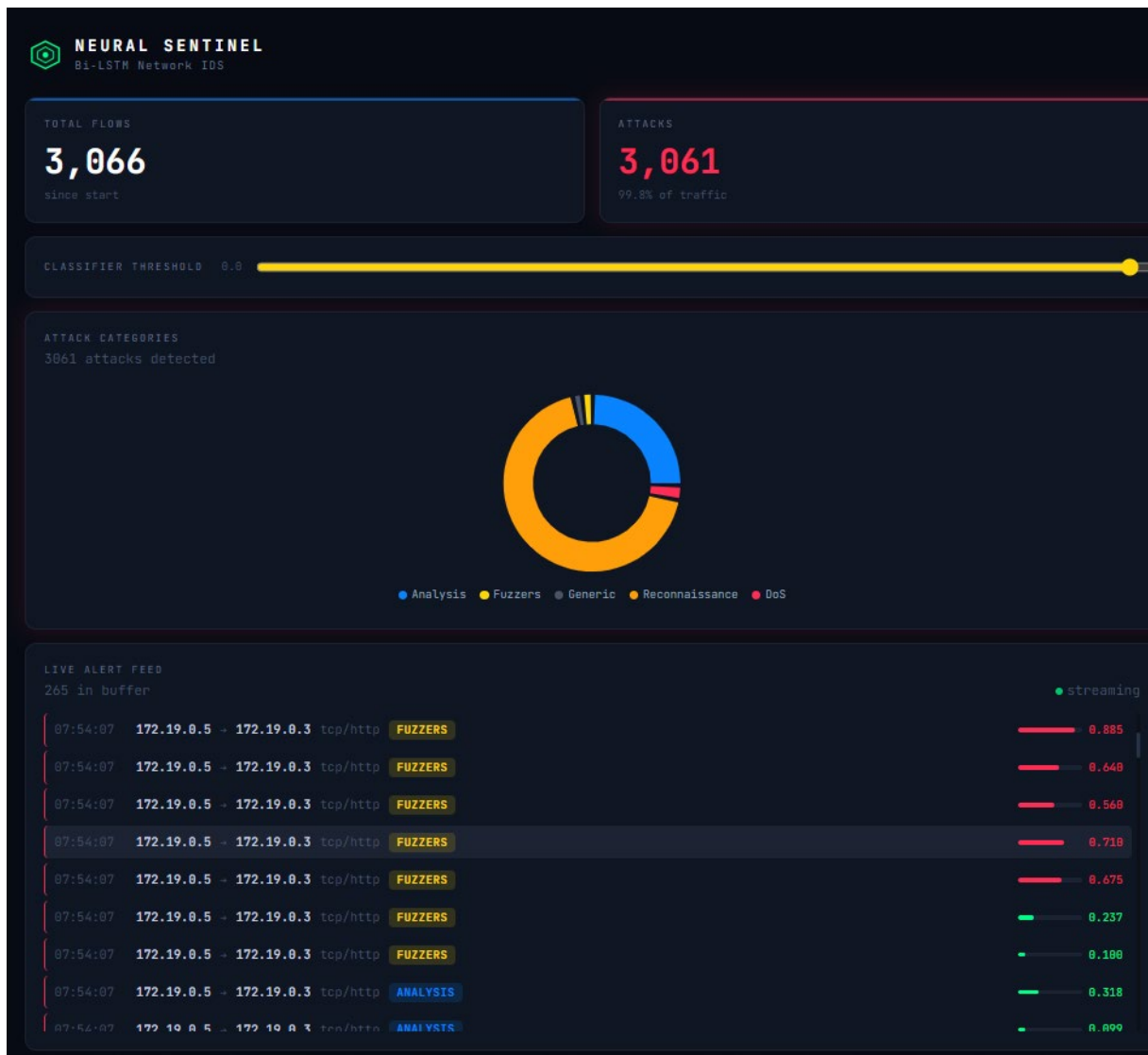


Figure 5.3a Detection of Fuzzing attacks by sentinel

```

root@3e107e66e826:/scripts# /scripts/run_fuzz.sh
[*] HTTP fuzzer → http://sentinel:8000
    404 responses are expected – they confirm the fuzzer is probing correctly.
    [phase 1] method probe (GET/POST/PUT/DELETE/HEAD/OPTIONS/PATCH/TRACE)
    POST → 405
    HEAD → 405
    DELETE → 405
    PUT → 405
    PATCH → 405
    GET → 200
    OPTIONS → 405
    TRACE → 405
    [phase 2] GET path brute-force (20 paths)
    *****
    * Wfuzz 3.1.0 - The Web Fuzzer *
    *****
  
```

Target: http://sentinel:8000/FUZZ
Total requests: 20

=====					
ID	Response	Lines	Word	Chars	Payload
=====					
000000001:	404	0 L	2 W	22 Ch	"admin"
000000003:	404	0 L	2 W	22 Ch	"backup"
000000007:	404	0 L	2 W	22 Ch	"config"
000000015:	404	0 L	2 W	22 Ch	".git"
000000020:	404	0 L	2 W	22 Ch	"token"
000000018:	404	0 L	2 W	22 Ch	"secrets"
000000014:	404	0 L	2 W	22 Ch	"cgi-bin"
000000019:	404	0 L	2 W	22 Ch	"keys"
000000017:	404	0 L	2 W	22 Ch	"shadow"
000000016:	404	0 L	2 W	22 Ch	"passwd"
000000006:	404	0 L	2 W	22 Ch	"api"
000000012:	404	0 L	2 W	22 Ch	"shell"
000000013:	404	0 L	2 W	22 Ch	"upload"
000000011:	404	0 L	2 W	22 Ch	"debug"
000000010:	404	0 L	2 W	22 Ch	"staging"
000000009:	404	0 L	2 W	22 Ch	"test"
000000008:	404	0 L	2 W	22 Ch	".env"
000000002:	404	0 L	2 W	22 Ch	"login"
000000004:	404	0 L	2 W	22 Ch	"phpmyadmin"
000000005:	404	0 L	2 W	22 Ch	"wp-admin"

Total time: 0.030607
Processed Requests: 20
Filtered Requests: 0
Requests/sec.: 653.4303

[phase 3] POST sweep

* Wfuzz 3.1.0 - The Web Fuzzer *

Target: http://sentinel:8000/FUZZ
Total requests: 20

=====					
ID	Response	Lines	Word	Chars	Payload
=====					
000000003:	404	0 L	2 W	22 Ch	"backup - backup"
000000014:	404	0 L	2 W	22 Ch	"cgi-bin - cgi-bin"
000000007:	404	0 L	2 W	22 Ch	"config - config"
000000020:	404	0 L	2 W	22 Ch	"token - token"
000000015:	404	0 L	2 W	22 Ch	".git - .git"

```

000000001: 404      0 L      2 W      22 Ch      "admin - admin"
000000019: 404      0 L      2 W      22 Ch      "keys - keys"
000000017: 404      0 L      2 W      22 Ch      "shadow - shadow"
000000018: 404      0 L      2 W      22 Ch      "secrets - secrets"
000000016: 404      0 L      2 W      22 Ch      "passwd - passwd"
000000006: 404      0 L      2 W      22 Ch      "api - api"
000000012: 404      0 L      2 W      22 Ch      "shell - shell"
000000013: 404      0 L      2 W      22 Ch      "upload - upload"
000000011: 404      0 L      2 W      22 Ch      "debug - debug"
000000010: 404      0 L      2 W      22 Ch      "staging - staging"
000000009: 404      0 L      2 W      22 Ch      "test - test"
000000008: 404      0 L      2 W      22 Ch      ".env - .env"
000000002: 404      0 L      2 W      22 Ch      "login - login"
000000004: 404      0 L      2 W      22 Ch      "phpmyadmin -
phpmyadmin"
000000005: 404      0 L      2 W      22 Ch      "wp-admin - wp-admin"

Total time: 0.031933
Processed Requests: 20
Filtered Requests: 0
Requests/sec.: 626.2959

[*] Fuzzing complete.
root@3e107e66e826:/scripts#

```

Figure 5.3b Logs from a 3 phased fuzzer strategy

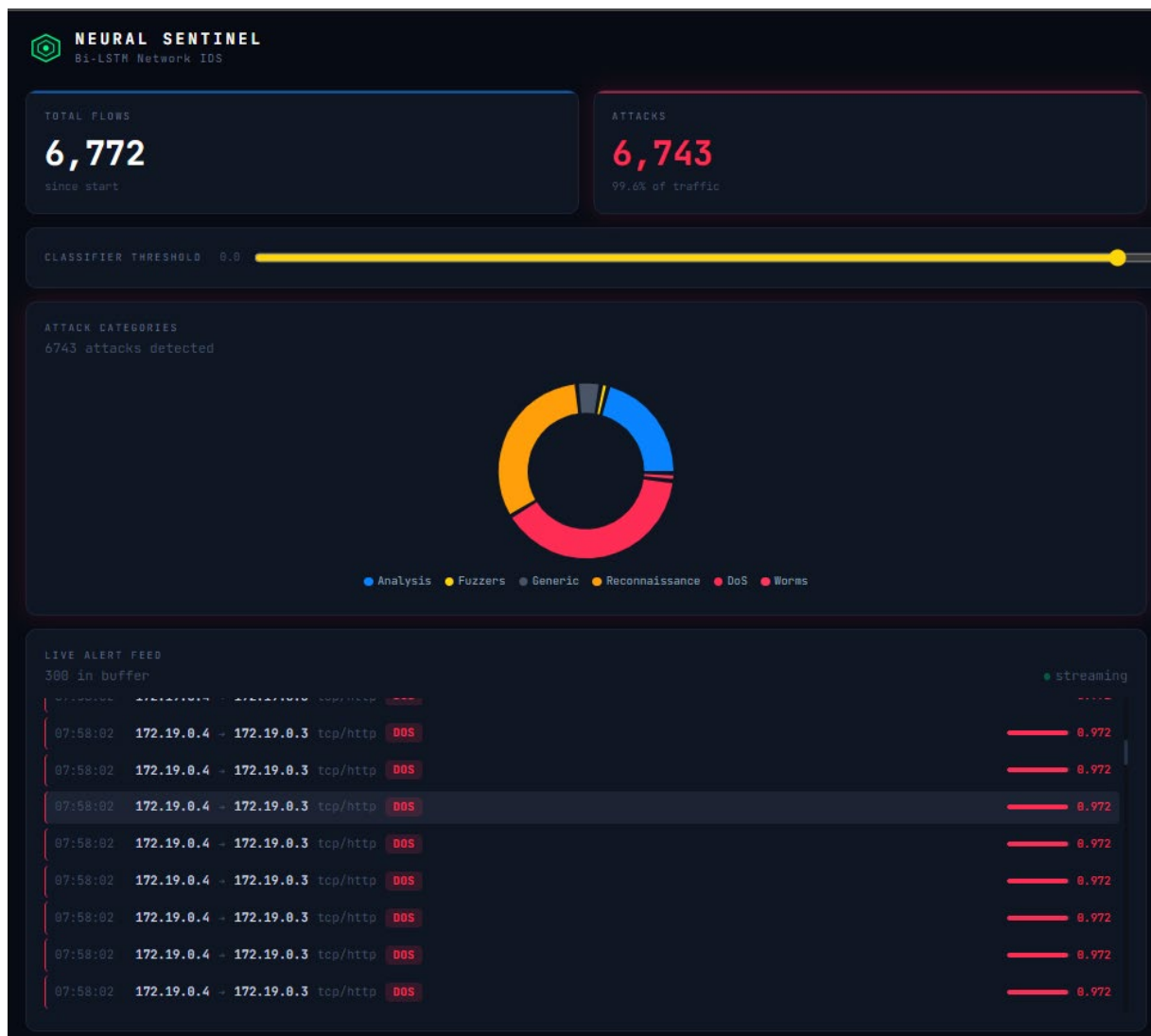


Figure 5.4a Detection DOS(denial of service) attack on Sentinel

```
root@3e107e66e826:/scripts# ./scripts/run_dos.sh
[*] SYN flood -> sentinel:8000 for 20s
    (hping3 --flood: 1 SYN pkt per flow, no ACK reply expected)
HPING sentinel (eth0 172.19.0.3): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown

--- sentinel hping statistic ---
752019 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
[*] DoS complete.
root@3e107e66e826:/scripts#
```

Figure 5.4b Logs from a hping script simulating DOS

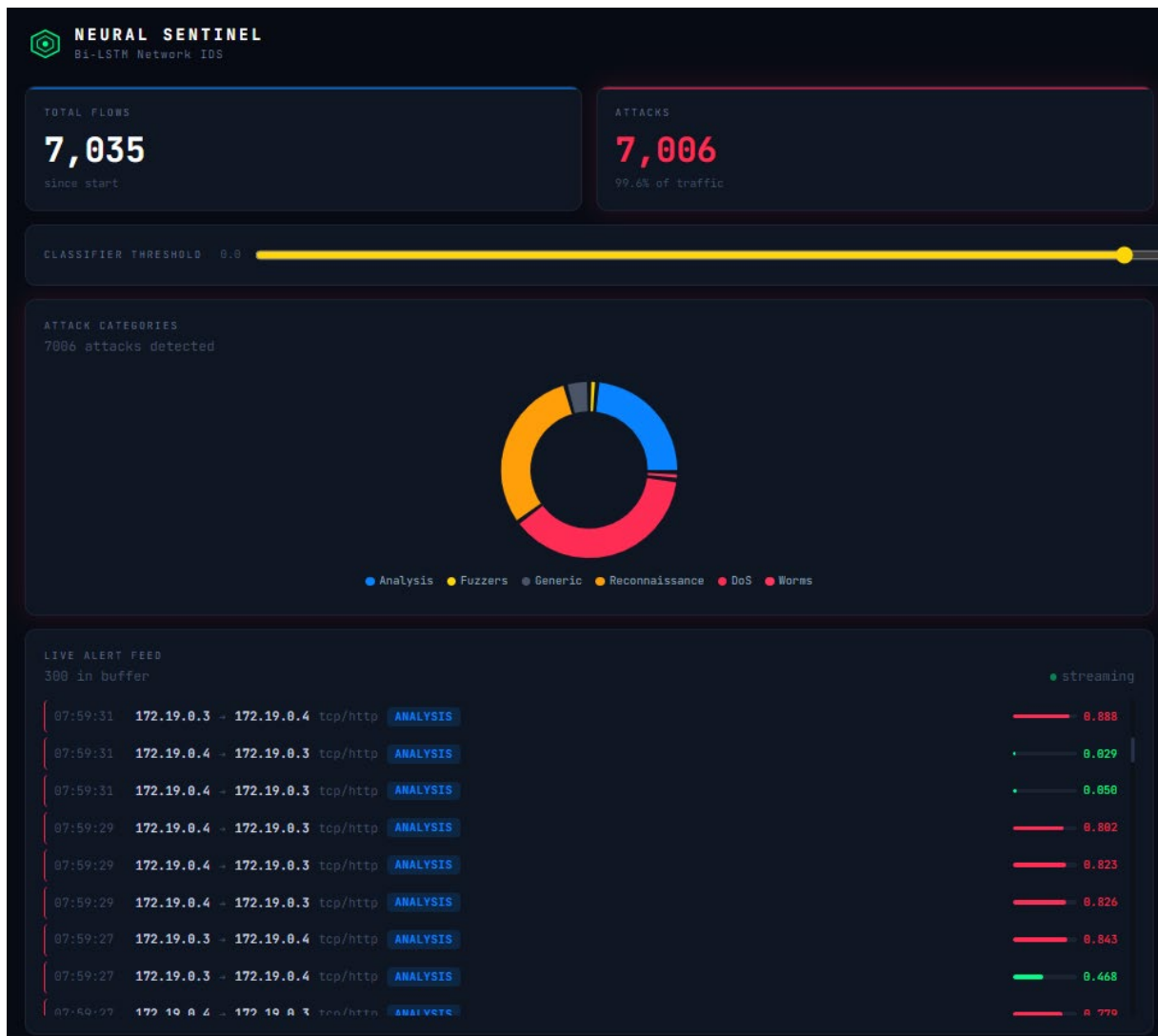


Figure 5.5a Detection Shellcode attack on Sentinel

Note how the sentinel marks it as “exploits” as high data flow is being handled easily by powerful machines but routers and such lo level devices would not be able to buffer it

```
[*] Shellcode delivery simulation → http://sentinel:8000
    Sending 12 x 4 KB payloads over separate TCP connections
    flow 11 → 200 (8192 B sent)
    flow 8 → 200 (8192 B sent)
    flow 1 → 200 (8192 B sent)
    flow 7 → 200 (8192 B sent)
    flow 6 → 200 (8192 B sent)
    flow 5 → 200 (8192 B sent)
    flow 10 → 200 (8192 B sent)
    flow 3 → 200 (8192 B sent)
    flow 2 → 200 (8192 B sent)
    flow 12 → 200 (8192 B sent)
    flow 9 → 200 (8192 B sent)
    flow 4 → 200 (8192 B sent)
[*] Shellcode simulation complete.
root@3e107e66e826:/scripts#
```

Figure 5.5b Logs from a script simulating shellcode

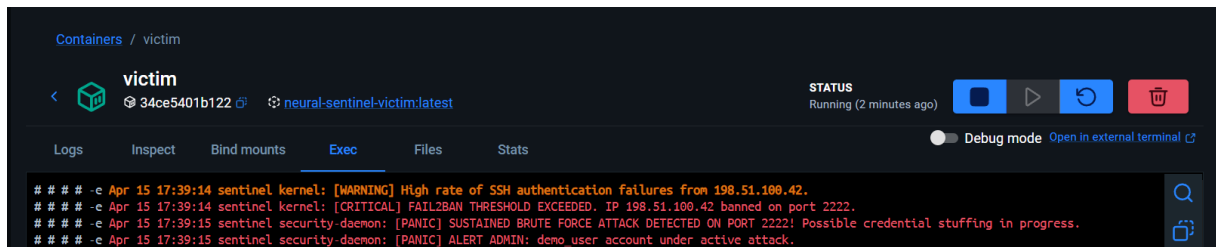


Figure 5.6a Detection and blocking of SSH brute force attacks

```

root@3e107e66e826:/scripts# ./scripts/run_ssh_brute.sh
[*] SSH brute force → demo_user@sentinel:2222
    Trying all 12 passwords – correct one is #11, so ≥10 flows are generated
Hydra v9.2 (c) 2021 by van Hauser/THC & David Maciejak - Please do not use in military or se
cret service organizations, or for illegal purposes (this is non-binding, these *** ignore l
aws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-15 08:00:10
[DATA] max 4 tasks per 1 server, overall 4 tasks, 12 login tries (l:1/p:12), ~3 tries per ta
sk
[DATA] attacking ssh://sentinel:2222/
[ATTEMPT] target sentinel - login "demo_user" - pass "password" - 1 of 12 [child 0] (0/0)
[ATTEMPT] target sentinel - login "demo_user" - pass "123456" - 2 of 12 [child 1] (0/0)
[ATTEMPT] target sentinel - login "demo_user" - pass "admin" - 3 of 12 [child 2] (0/0)
[ATTEMPT] target sentinel - login "demo_user" - pass "toor" - 4 of 12 [child 3] (0/0)
[ATTEMPT] target sentinel - login "demo_user" - pass "root" - 5 of 12 [child 1] (0/0)
[ATTEMPT] target sentinel - login "demo_user" - pass "letmein" - 6 of 12 [child 0] (0/0)
[ATTEMPT] target sentinel - login "demo_user" - pass "password123" - 7 of 12 [child 2] (0/0)
[ATTEMPT] target sentinel - login "demo_user" - pass "qwerty" - 8 of 12 [child 3] (0/0)
[ATTEMPT] target sentinel - login "demo_user" - pass "12345678" - 9 of 12 [child 1] (0/0)
[ATTEMPT] target sentinel - login "demo_user" - pass "welcome" - 10 of 12 [child 2] (0/0)
[ATTEMPT] target sentinel - login "demo_user" - pass "master" - 11 of 12 [child 3] (0/0)
[ATTEMPT] target sentinel - login "demo_user" - pass "s3ntinel_brute" - 12 of 12 [child 0] (
0/0)
[2222][ssh] host: sentinel login: demo_user password: s3ntinel_brute
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-15 08:00:22
[*] SSH brute force complete

```

Figure 5.6b Using Hydra to simulate brute force password guessing attacks

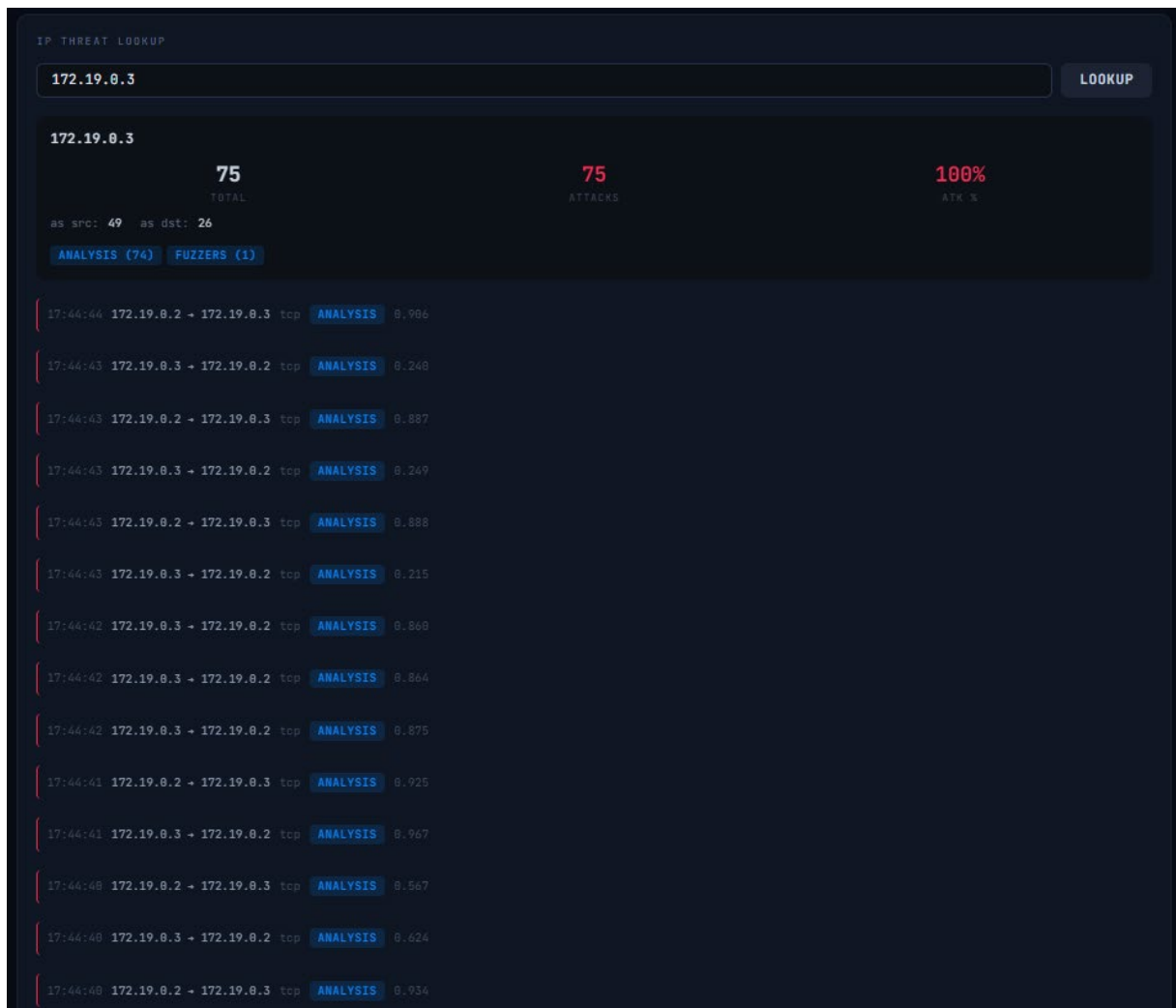


Figure 5.7 IP lookup feature for source-based threat monitoring

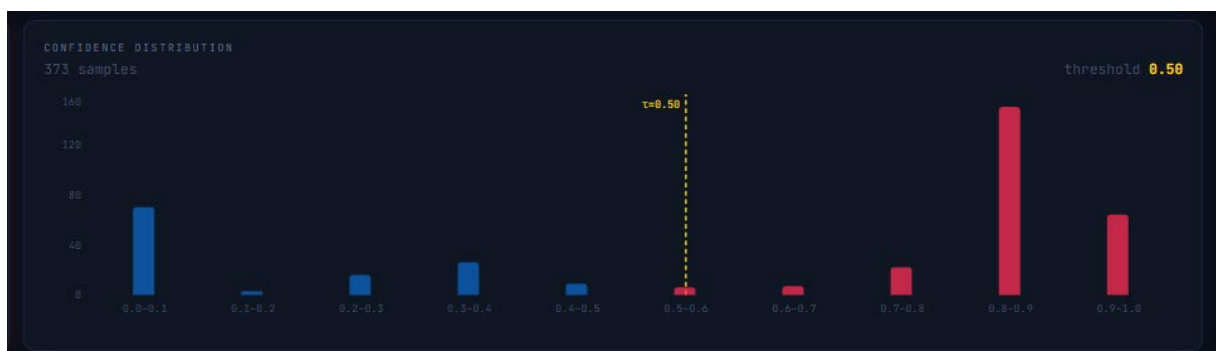


Figure 5.8 Confidence distribution graph for live traffic

Chapter 6

Conclusion

This project presents Neural Sentinel, a decentralized intrusion detection system that combines deep learning and blockchain technology to address the growing security challenges in modern network environments.

By shifting away from centralized data processing, the implementation of Federated Learning (FedAvg) allows multiple network nodes to collaboratively train a global Bidirectional LSTM model. This architecture successfully captures complex, multi-stage sequential attack patterns while strictly preserving the privacy of local organizational data.

The experimental results validate our core engineering philosophy: prioritizing security analyst bandwidth. By achieving 97% to 100% precision across benchmark datasets like UNSW-NB15 and NSL-KDD, the system decisively mitigates the alert fatigue common in anomaly-based systems. While the conservative classification threshold results in a moderate recall rate, this intentional trade-off ensures that all generated alerts represent highly credible threats.

Furthermore, the integration of a Tamper-Evident Local Blockchain addresses a critical vulnerability in incident response. By cryptographically chaining detection logs via SHA-256 hashing, the system guarantees the immutable forensic integrity of the audit trail against post-breach manipulation. Packaged within a scalable Docker microservices ecosystem and monitored via a real-time React dashboard, Neural Sentinel proves that advanced, federated

deep learning can be practically deployed to secure modern, decentralized network infrastructures.

The system's practical design considerations make it suitable for real-world deployment. With real-time detection capabilities and response times under 50 milliseconds, the system can operate effectively in production environments. The modular architecture allows for independent development, testing, and optimization of components, supporting maintainability and future enhancements.

The comprehensive handling of class imbalance through optimized SMOTE ensures robust detection of both common and rare attack types. This addresses a critical limitation of existing systems that often perform poorly on minority attack classes. The bidirectional processing of network traffic captures complete temporal context, enabling the identification of complex attack patterns that span multiple time steps.

Future work will focus on several key areas. The system can be extended to support additional datasets and attack types, enhancing its generalization capabilities. Integration with advanced blockchain platforms and consensus mechanisms could improve scalability and performance. The development of interpretability tools will help security analysts understand detection decisions and validate system behavior. Finally, deployment in real production networks will provide valuable insights for further optimization and refinement.

In conclusion, Neural Sentinel represents a significant advancement in intrusion detection technology, offering a practical, high-performance solution that addresses the limitations of existing systems. By combining state-of-the-art deep learning techniques with blockchain-based security mechanisms, the system provides accurate, trustworthy, and adaptable protection for modern network infrastructure. The project demonstrates that it is possible to achieve both exceptional detection accuracy and practical deployability, making advanced intrusion detection accessible to organizations of all sizes.

Appendix

List of containers demonstrating IDS in action

☐	▼	●	neural-sentinel	-	-	-	20.24%	304.2MB / 30.4G	3.91%	975KB / 1.			
☐		●	sentinel-ui	6dcbf5bf678a	neural-seni	3000:80 ↗	0.2%	15.66MB / 7.6GB	0.2%	0B / 8.19K			
☐		●	attacker	3e107e66e826	neural-seni		0%	1.62MB / 7.6GB	0.02%	975KB / 0			
☐		●	sentinel	8e2b9fab945c	neural-seni	2222:2222 ↗ Show all ports (2)	20.04%	284.9MB / 7.6GB	3.66%	0B / 1.04M			
☐		●	victim	2eda9a4240e4	neural-seni		0%	2.02MB / 7.6GB	0.03%	0B / 4.1KB			

References

- [1] Bajpai SA, Patankar AB. Self-configuring intrusion detection using adaptive goal target optimization based deep Bi-LSTM in blockchain networking systems. *Intelligent Decision Technologies*. 2025;19(1):175-193. doi:10.1177/18724981251330736
- [2] M. Kumar and A. K. Singh, "Distributed Intrusion Detection System using Blockchain and Cloud Computing Infrastructure," 2020 4th International Conference on Trends in Electronics and Informatics (ICOEI)(48184), Tirunelveli, India, 2020, pp. 248-252, doi: 10.1109/ICOEI48184.2020.9142954.
- [3] V. Tomar and P. S. Mehra, "Deep Learning Bi-LSTM Model for Intrusion Detection in IoT," 2023 5th International Conference on Advances in Computing, Communication Control and Networking (ICAC3N), Greater Noida, India, 2023, pp. 1342-1347, doi: 10.1109/ICAC3N60023.2023.10541673.
- [4] K. RadhaKrishna, C. R. Babu, D. Abraham, P. K. Abhilash, H. M. Abbas and N. M, "Cyber Resilience in Smart Cities Using Bi-LSTM-Based Intrusion Detection System," 2025 International Conference on Computational Innovations and Engineering Sustainability (ICCIES), Coimbatore, Tamilnadu, India, 2025, pp. 1-7, doi: 10.1109/ICCIES63851.2025.11033050.
- [5] NSL-KDD Dataset. Available: <https://www.kaggle.com/datasets/sampadab17/network-intrusion-detection>
- [6] UNSW-NB15 Dataset. Available: <https://research.unsw.edu.au/projects/unsw-nb15-dataset>
- [7] BoT-IoT Dataset. Available: <https://research.unsw.edu.au/projects/bot-iot-dataset>
- [8] O. Alkadi, N. Moustafa, B. Turnbull and K.-K. R. Choo, "A Deep Blockchain Framework-Enabled Collaborative Intrusion Detection for Protecting IoT and Cloud Networks," *IEEE Internet of Things Journal*, vol. 8, no. 12, pp. 9463-9472, 15 June 2021, doi: 10.1109/JIOT.2020.3008950.

- [9] S. K. Poorazad, C. Benzaid and T. Taleb, "Blockchain and Deep Learning-Based IDS for Securing SDN-Enabled Industrial IoT Environments," 2023 IEEE Global Communications Conference (GLOBECOM), Kuala Lumpur, Malaysia, 2023, pp. 2760-2765, doi: 10.1109/GLOBECOM54140.2023.10436839.
- [10] H. Alamro et al., "Modeling of Blockchain Assisted Intrusion Detection on IoT Healthcare System Using Ant Lion Optimizer With Hybrid Deep Learning," IEEE Access, vol. 11, pp. 82200-82212, 2023, doi: 10.1109/ACCESS.2023.3294318.
- [11] A. Ravuri, "Blockchain-enabled collaborative anomaly detection for IoT security," Future Generation Computer Systems, 2024, doi: 10.1016/j.future.2024.01.035.
- [12] A. Derhab, M. Guerroumi, M. Belaoued, and O. Cheikhrouhou, "Blockchain and Random Subspace Learning-Based IDS for SDN-Enabled Industrial IoT Security," Sensors, vol. 19, no. 14, 2019, doi: 10.3390/s19143119.
- [13] M. Islam, A. Rahman, et al., "Blockchain-SDN-Based Energy-Aware and Distributed Secure Architecture for IoT in Smart Cities," IEEE Internet of Things Journal, vol. 7, no. 4, pp. 3208-3222, 2020, doi: 10.1109/JIOT.2020.2973418.

Acknowledgements

We extend our deepest appreciation to our supervisor, Mr. Vaibhav Ambhire, whose scholarly guidance and unwavering commitment were instrumental in the realization of our project, “Neural Sentinel: A Decentralized, Deep Learning-based Intrusion Detection Solution”. His profound expertise in the field and infectious enthusiasm for innovation served as a constant catalyst for our progress. We are profoundly indebted to him for instilling in us the analytical discipline and technical precision necessary to navigate the complexities of this research; working under his tutelage has been an invaluable cornerstone of our academic journey.

We thank our Head of Department, Dr. Tanuja Sarode, for her strategic oversight and we owe a debt of gratitude to Dr Archana Patankar for inspiring this idea in the first place. We also acknowledge the collective contributions of the faculty and technical staff who provided the intellectual environment required to translate abstract theories into a tangible architecture.

On a personal note, this milestone could not have been reached without the steadfast encouragement of our families and peers. Their resilience, perspective, and emotional support provided the necessary balance during the most demanding phases of this endeavor. To all those who offered their assistance, whether through technical discourse or quiet encouragement, we offer our most sincere thanks for helping bring this vision to fruition.

Gauresh Bagayatkar

Yash Shelar

Ocean Chaudhary

Vedant Chaudhari